

Profiling and Reducing Linux Kernel Boot Time on Resource-Constrained Embedded Boards

A Practitioner's Guide (Raspberry Pi)

Kartikey Dubey

*Department of Computer Science and
Engineering*

*International Institute of Information
Technology Bangalore*

Bengaluru, India

Kartikey.Dubey@iiitb.ac.in

B. Thangaraju

*Department of Computer Science and
Engineering*

*International Institute of Information
Technology Bangalore*

Bengaluru, India

b.thangaraju@iiitb.ac.in

Abstract

This report documents a systematic, stage-by-stage reduction of Linux boot time on a Raspberry Pi 4 Model B, covering both kernel-level and userspace-level optimization for a Headless Server profile (Raspberry Pi OS Lite) and an Interactive Unit profile (Raspberry Pi OS with Desktop). On the kernel side, disabling the tracing subsystem, the VideoCore/VCHIQ display-and-media stack, and the USB/PCIe subsystem reduced real-world kernel initialization time from approximately 2.0s (stock) to approximately 0.81s for the Headless profile — a reduction of roughly 59% — while an instrumented initcall-level investigation additionally uncovered and root-caused an upstream kernel lock-contention bug affecting tracing-related initcalls. On the userspace side, disabling the `cloud-init` provisioning service group produced the largest early improvement to time-to-target; for the Headless profile, switching from NetworkManager to a lighter network stack (`systemd-networkd` with `wpa_supplicant`), together with explicit exclusion of an unused Ethernet interface, reduced total boot time to approximately 6.01s after the subsequent optional-service cleanup. The Interactive Unit profile retains NetworkManager for its GUI configuration convenience. Both profiles are pinned to kernel version `6.18.34+rpt-rpi-v8` for internal consistency.

Contents

1	Project Definition and Scope	2
2	Hardware and Software Setup	2
2.1	Hardware Specification	2
2.2	Software / OS Choice	2
3	Methodology	3
3.1	Userspace and Overall Boot Time Measurement	3
3.2	Kernel-Internal Timing Measurement	3
3.3	Data Compilation and Visualization	3
4	Baseline Boot Time	4
5	Optimizing the Kernel	4
5.1	Methodology: Building from the Raspberry Pi Kernel Fork	5
5.2	Basic Optimization: Removing the Tracing Subsystem	6
5.3	Root Cause: Mutex Contention in Tracing Initcalls	7

6	Two Optimization Paths	8
7	Kernel-Level Optimization per Path	9
7.1	Kernel A: Headless Server	9
7.2	Kernel B: Interactive Unit	13
8	Userspace Optimization	14
8.1	Headless Server	15
8.2	Interactive Mode	23
9	Final Results	30
9.1	Headless Server: Optimized Kernel + Userspace	30
9.2	Interactive Unit: Optimized Kernel + Userspace	30
10	Conclusion	31
10.1	Future Work	32
11	References	33

1 Project Definition and Scope

A systematic removal of unused kernel subsystems combined with deferred module initialization and initcall reordering. Boot time is measured using `systemd-analyze` before and after each optimization stage, producing a reproducible methodology that any embedded team can apply.

For the current iteration of this work, scope is restricted to the **Raspberry Pi** platform only.

2 Hardware and Software Setup

2.1 Hardware Specification

- Board: Raspberry Pi 4 Model B (Rev 1.4)
- SD Card: **Samsung EVO Plus 64 GB microSDXC**, Class 10, read speed up to 100–130 MB/s, write speed approximately 20–60 MB/s.
- Power Supply: **Official Raspberry Pi 15.3W USB-C Power Supply (5.1V DC, 3.0A)**.

2.2 Software / OS Choice

Two operating system images are used, one per target deployment profile under investigation:

- **Raspberry Pi OS Lite** — used as the base image for the *Headless Server* profile. Lite excludes the desktop environment, windowing system, and associated userspace daemons entirely, making it the appropriate starting point for a deployment with no display, audio, or interactive requirements.
- **Raspberry Pi OS (with Desktop)** — used as the base image for the *Interactive Unit* profile, where a graphical session, audio, and Bluetooth peripheral support are treated as functional requirements.

Both images are officially maintained and supported by the Raspberry Pi Foundation, which was a deciding factor in their selection: official support ensures continued compatibility with board-specific firmware, GPU driver stacks, and kernel updates, and makes the resulting methodology directly reproducible by readers using standard, unmodified tooling (Raspberry Pi Imager) rather than a community-maintained or hand-built distribution.

Critically, both images are built on the **Raspberry Pi Foundation’s fork of the Linux kernel (`raspberrypi/linux`)** rather than an unmodified upstream kernel. This is not a stylistic choice but a hardware necessity: the Raspberry Pi 4’s HDMI output, VideoCore GPU integration, and several other SoC-specific peripherals rely on proprietary firmware interfaces and companion drivers (e.g., `vc4-kms-v3d`, `VCHIQ`) that are maintained and shipped exclusively through this fork. An unmodified mainline kernel would not correctly initialize these components, making the Raspberry Pi fork the only viable baseline for kernel-level optimization work on this hardware.

Both OS variants share an identical underlying kernel package; the two images diverge entirely in userspace composition (desktop environment, compositor, audio stack, and related services). This distinction directly motivates the project’s later split into independent kernel-level (Section 7) and userspace-level (Section 8) optimization tracks.

3 Methodology

Two complementary instrumentation approaches are used throughout this project: one for overall/userspace boot timing, and one for kernel-internal timing. Both are automated via a systemd simple service paired with a shell script, so that each measurement stage can be repeated unattended across multiple boot cycles.

3.1 Userspace and Overall Boot Time Measurement

Overall boot time — and its breakdown into kernel and userspace phases — is measured using **systemd-analyze**. A dedicated systemd service invokes a custom shell script on every boot; the script waits for the system to reach a stable running state, then captures the following for each of **10 iterations**:

- **systemd-analyze time** — reports total boot time, the kernel/userspace time split, and the elapsed time at which each systemd target (e.g., `multi-user.target`, `graphical.target`) was reached.
- **systemd-analyze blame** — reports the individual initialization time of each systemd service/unit, sorted by duration.
- **systemd-analyze critical-chain** — identifies the specific chain of services that block progress toward the target boot goal, distinguishing services on the critical path from those that merely run in parallel without delaying boot completion.

After each iteration's data is captured, the script triggers a reboot and re-invokes itself automatically, repeating until all 10 iterations are collected. Once the tenth iteration completes, the script disables its own systemd service, so that the device returns to a normal boot cycle without further automatic reboots once the measurement batch is finished.

3.2 Kernel-Internal Timing Measurement

To measure where time is spent during kernel initialization specifically, the kernel command line (`cmdline.txt`) is augmented with the `initcall_debug`, `printk.time=1`, and `loglevel=8` parameters. These cause the kernel to emit a timestamped entry for every initcall's invocation and completion, including its duration, directly into the kernel ring buffer.

Since these debug parameters add logging overhead not present in a production boot (Section 5.2), and since this study also requires clean, uninstrumented boots for real-world timing comparisons, a dedicated shell script manages the boot command-line parameters in `cmdline.txt` rather than editing the file by hand for each measurement pass. When instrumentation is enabled, the shell script appends the required parameters to `cmdline.txt`; when it is disabled, the script removes exactly those parameters while leaving the rest of the command line untouched. This keeps the instrumented and clean measurement conditions reproducible and avoids undocumented manual changes between runs.

Mirroring the userspace approach, a second systemd service invokes a shell script that captures the `dmesg` output to a log file on each boot, across 10 iterations, before triggering the next reboot.

3.3 Data Compilation and Visualization

In both cases, the resulting raw text logs (10 runs each) are parsed by dedicated Python scripts into structured CSV data, from which mean, standard deviation, minimum, and maximum values are computed per service or per initcall across all runs. This aggregated data is then

used to generate the comparison charts presented throughout this report (bar charts of top time-consuming components and stacked kernel-vs-userspace summaries).

Note: kernel-internal measurement (`initcall_debug`) necessarily introduces its own logging overhead, discussed further in Section 4.1.

4 Baseline Boot Time

Combined kernel and userspace total boot time was captured for the stock, pre-optimization configuration of both profiles, as the starting point for all subsequent work:

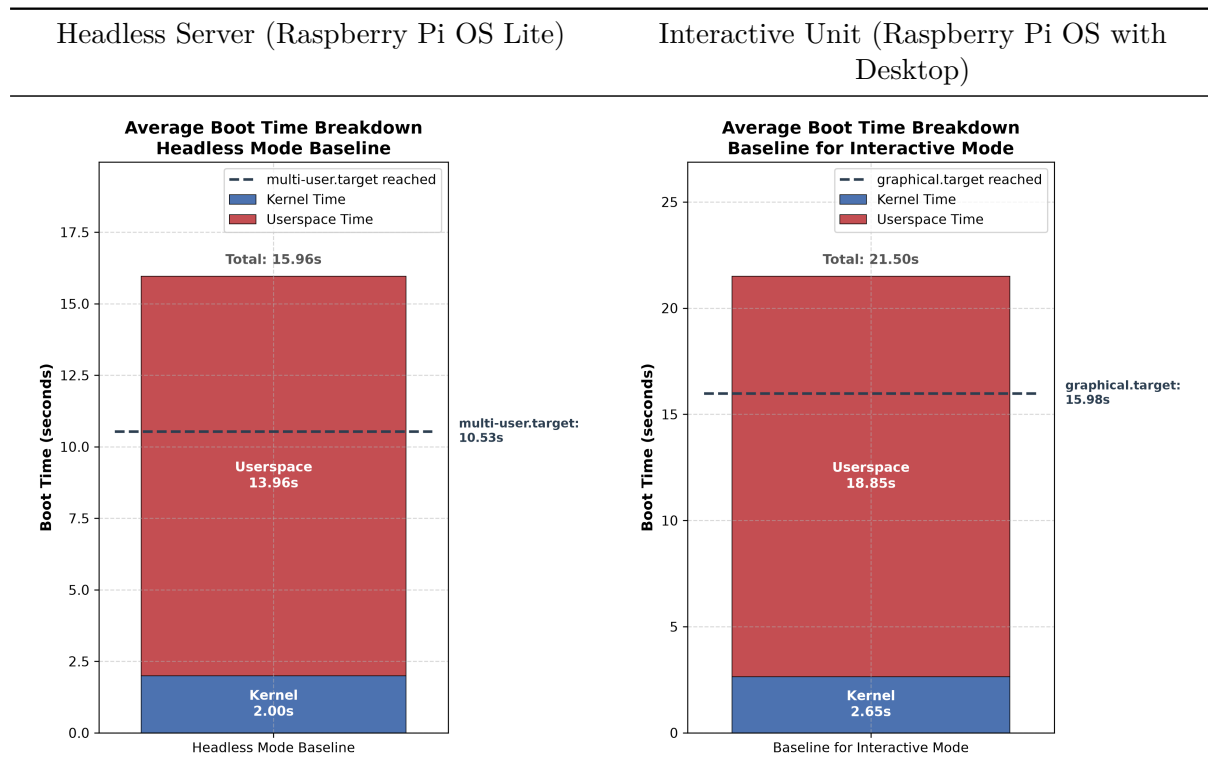


Table 1: Stock, pre-optimization total boot time — Headless Server versus Interactive Unit.

5 Optimizing the Kernel

With the overall baseline established (Section 4), the investigation proceeds first into kernel-level behavior — profiling and optimizing the kernel’s own initialization sequence before turning to userspace (Section 8). The starting point is the stock kernel’s initcall breakdown:

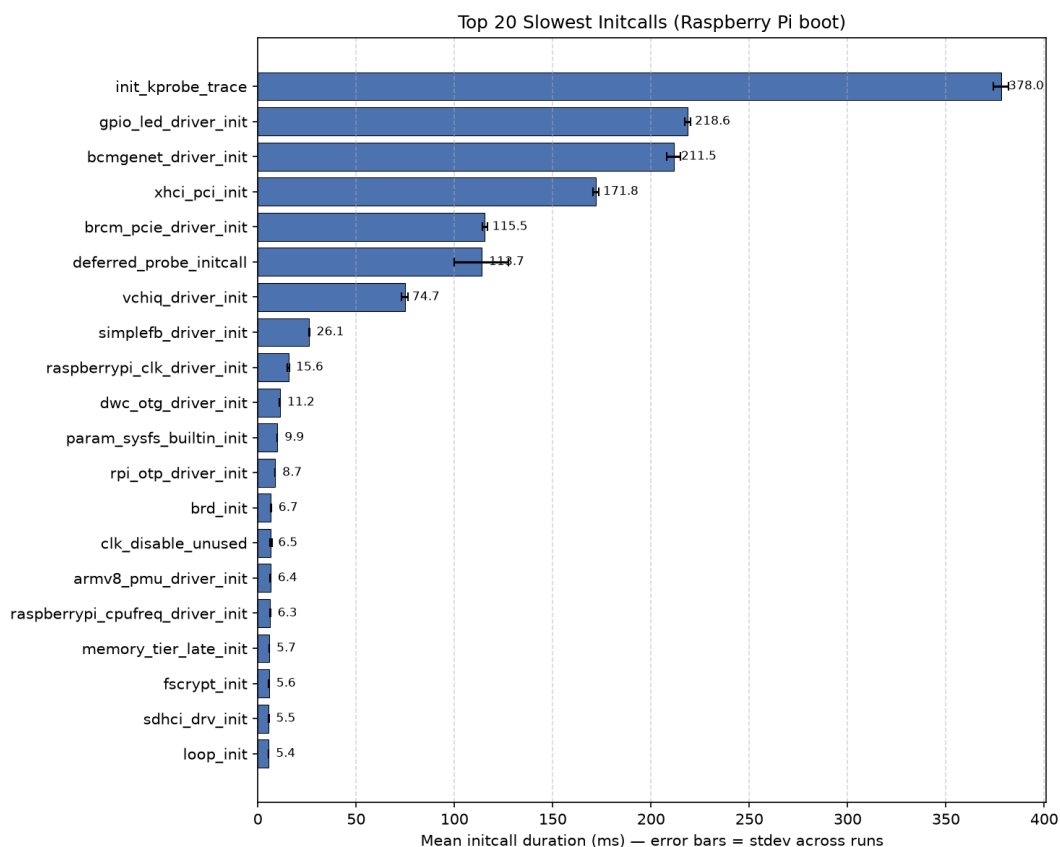


Figure 1: Baseline kernel initcall timing — top slowest subsystems (stock kernel).

Note: this measurement includes instrumentation overhead from `initcall_debug/printk.time` logging and serial console output (Section 3.2), and is not representative of production boot time on its own — see Section 7.1 for the corresponding clean-cmdline, real-world figures.

5.1 Methodology: Building from the Raspberry Pi Kernel Fork

All kernel-level modifications in this project are built from source using the official Raspberry Pi Foundation kernel repository, `raspberrypi/linux`, rather than an unmodified upstream (kernel.org) tree — for the reasons outlined in Section 2.2. The build process is designed to be repeatable across optimization stages, so that each stage differs from the previous by exactly one configuration change (see Section 7).

Source and Branch Selection

The kernel source is cloned as a shallow, single-branch checkout to minimize download size, using the branch corresponding to the major/minor version of the kernel already running on the target device (6.18.34+rpt-rpi-v8):

```
git clone --depth=1 --branch rpi-6.18.y https://github.com/raspberrypi/linux.git
cd linux
```

The Raspberry Pi kernel fork follows an `rpi-X.Y.y` branch naming convention rather than exposing individual patch-level tags matching `uname -r` directly; exact patch-level parity with the running kernel is not required for this study, since all optimization stages are built from, and compared against, the same cloned source tree — preserving internal consistency of the before/after measurements regardless of minor version drift from the shipped image.

Cross-Compilation Toolchain

Builds are performed on a separate x86_64 host machine rather than natively on the Raspberry Pi, owing to the significant build-time advantage of a desktop-class multi-core CPU over the target board's own processor. The following environment variables direct the kernel build system to use the ARM64 cross-compiler:

```
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
```

Baseline Configuration

Each build stage begins from the board-specific default configuration provided upstream for the Raspberry Pi 4 / 400 / Compute Module 4 family:

```
make bcm2711_defconfig
```

Stage-specific configuration options are then toggled using the `scripts/config` helper utility (e.g., `scripts/config --disable CONFIG_FTRACE`), followed by `make olddefconfig` to resolve any dependent configuration options and ensure a consistent, buildable `.config`. Each resulting configuration is preserved (e.g., `configs/stage1_drm_disabled.config`) so that later stages can be built cumulatively or independently as required, without needing to manually re-apply prior changes.

Build and Deployment

The kernel image, loadable modules, and device tree blobs are built with:

```
make -j$(nproc) Image modules dtbs
```

The resulting `Image` file, corresponding `.dtb` for the target board revision, and kernel modules are then copied to the boot and root partitions of a dedicated test SD card. A selectable dual-kernel boot configuration (via the `kernel=` directive in `config.txt`) is used throughout, allowing the stock/known-working kernel to remain available as an immediate fallback without requiring a full re-flash between test iterations.

5.2 Basic Optimization: Removing the Tracing Subsystem

An initial pass through the baseline initcall data (Section 4.1, Figure 1) identified `init_kprobe_trace` as the single largest contributor to kernel initialization time, with a mean duration of approximately **376 ms** ($n = 10$, $\sigma \approx 3.9$ ms) — over four times larger than any other individual initcall observed. This function is part of the kernel's kprobe-based dynamic event tracing infrastructure, a debug/development feature not required for a production embedded deployment.

Note: all timing figures in this section (Section 5) are drawn from `initcall_debug`-instrumented captures (Section 3.2) and include logging/serial-console overhead not present in a production boot. They are reported here only to establish the relative magnitude and ranking of initcall costs between stages — not as representative real-world boot times. The corresponding real-world, clean-cmdline figures are presented separately in Section 7.1 and confirmed as tracking the same trend in Section 7.1.

Based on this finding, the following configuration options were disabled in the kernel's `.config`, using the baseline defconfig established in Section 5.1 as the starting point:

```
scripts/config --disable CONFIG_FTRACE
scripts/config --disable CONFIG_KPROBE_EVENTS
scripts/config --disable CONFIG_EVENT_TRACING
```

```
scripts/config --disable CONFIG_KPROBES
make olddefconfig
```

Following `olddefconfig` resolution, `CONFIG_KPROBES` was found to be automatically re-enabled as a dependency of `CONFIG_KGDB` (kernel debugger support), which declares an unconditional `select KPROBES` in `lib/Kconfig.kgdb`. This was verified to be inconsequential to the optimization target: `CONFIG_KPROBE_EVENTS` and `CONFIG_EVENT_TRACING` — the options directly responsible for the expensive tracing initialization path — remained successfully disabled, and the base `KPROBES` infrastructure alone does not invoke `init_kprobe_trace`.

The kernel was rebuilt and the 10-iteration `initcall_debug` capture repeated under identical conditions. Total mean instrumented `initcall` time across all runs fell by approximately **40%**, with `init_kprobe_trace` no longer appearing in the kernel log at all, confirming the affected code path was fully compiled out rather than merely optimized.

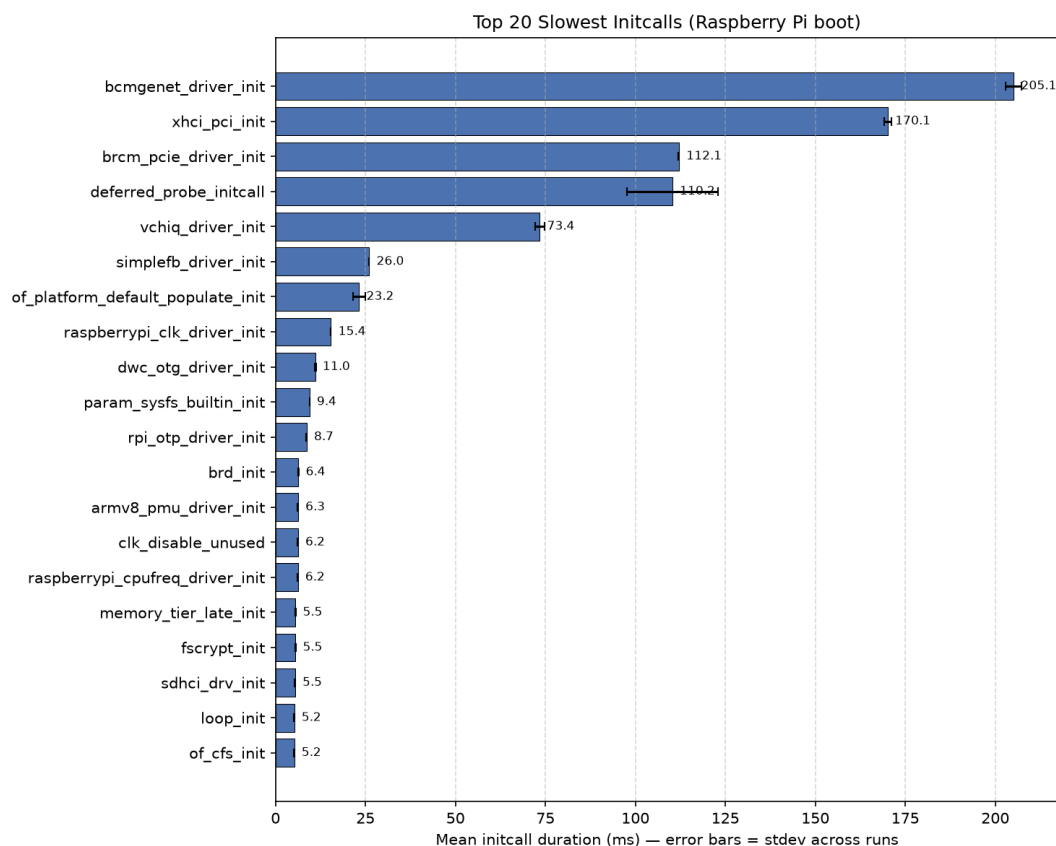


Figure 2: Kernel initcall timing after tracing subsystem removal.

5.3 Root Cause: Mutex Contention in Tracing Initcalls

The magnitude of `init_kprobe_trace`'s baseline cost (376 ms for what should be a lightweight registration routine) warranted further investigation before being accepted at face value. Cross-referencing kernel configuration (self-test options such as `CONFIG_FTRACE_STARTUP_TEST` were already disabled in the baseline, ruling out self-test overhead) and the surrounding `dmesg` context — which showed the entire delay occurring silently between the `calling` and `returned` log lines with no intervening kernel output — pointed to the function being *blocked*, rather than performing genuine work.

This was confirmed to correspond to a documented upstream kernel issue. `init_kprobe_trace` invokes `setup_boot_kprobe_events()`, whose child function `enable_boot_kprobe_events()`

requires the `event_mutex` lock. This lock is held for an extended period by an unrelated earlier initcall, `early_event_add_tracer()`, which is itself waiting on the `trace_event_sem` read-write semaphore — a semaphore held by `trace_event_update_all()` while it updates trace event metadata across the entire kernel. The result is a lock-contention chain in which `init_kprobe_trace` is not slow due to its own logic, but due to serialized waiting on infrastructure unrelated to kprobe events specifically.

An upstream patch introducing a `trace_async_init` boot parameter, which moves `setup_boot_kprobe_events()` onto an asynchronous workqueue (`trace_init_wq`) to avoid blocking the boot-critical path, was identified in kernel mailing list discussion dated January 2026¹. The kernel version used in this study (`rpi-6.18.y`) was confirmed, via configuration inspection, to predate this fix. Consequently, disabling the tracing subsystem entirely — as performed in Section 5.2 — was adopted as the practical mitigation for this deployment, since it avoids the contended code path altogether rather than requiring a backport of the asynchronous-initialization patch.

A comparison of the full pre- and post-optimization initcall datasets revealed that `gpio_led_driver_init` — responsible for registering GPIO-connected status LEDs (e.g., the board’s ACT LED) as standard Linux LED class devices — was also substantially affected by the same lock contention chain described above, despite having no functional relationship to kprobes or tracing: in the baseline measurement this initcall averaged approximately **218 ms**, falling to approximately **1.8 ms** following removal of the tracing subsystem, indicating the contention was serializing multiple, functionally unrelated initcalls rather than being isolated to `init_kprobe_trace` alone.

Note added during later work: subsequent testing on the Interactive Unit profile found `gpio_led_driver_init`’s presence in the initcall log also correlates with kernel point-version independently of tracing configuration — the initcall was present under kernel 6.18.34+rpt-rpi-v8 but absent under 6.18.39+rpt-rpi-v8, with tracing enabled in both cases. A configuration diff between the two point-versions’ `.config` files found no meaningful difference, so the root cause of this version-dependent behavior remains unidentified. Consequently, the improvement described above should be read as at least partially attributable to the lock-contention fix, with a version-dependent component not fully isolated by this study; see Future Work (Section 10).

6 Two Optimization Paths

The removals made so far (tracing subsystem, Section 5) apply unconditionally to any deployment and carry no functional trade-off. Beyond this point, further kernel-level reduction requires removing subsystems that are only unnecessary for *some* deployments — the VideoCore display stack and USB/PCIe being the clearest examples. Continuing as a single, one-size-fits-all optimization track would therefore force an arbitrary choice on behalf of the reader. Instead, optimization is split into two explicit target profiles, each with a stated, deliberately chosen set of requirements against which every subsequent removal decision is judged:

1. **Headless Server** — no display, audio, or Bluetooth required. Assumes network connectivity (wired or wireless) and, optionally, USB peripherals as the only interaction surface. This is the profile evaluated in this report.
2. **Interactive Unit** — requires HDMI output, audio, and Bluetooth, corresponding to a Raspberry Pi OS (Desktop) deployment. This profile is evaluated separately in Sections 7.2 and 8.2.

¹Linux kernel mailing list, `linux-trace-kernel`: patch series introducing `trace_async_init` to defer `setup_boot_kprobe_events()` onto an asynchronous workqueue, v4, January 2026. See References, Section 11.

Both profiles share the tracing-subsystem baseline from Section 5 and continue to use the same measurement methodology (Section 3) throughout.

7 Kernel-Level Optimization per Path

7.1 Kernel A: Headless Server

Building on the tracing-subsystem baseline established in Section 5 (common to both Kernel A and Kernel B), the headless profile additionally removes the VideoCore/display stack (CONFIG_DRM_VC4, CONFIG_BCM2835_VCHIQ and its audio/camera/codec/ISP dependents, per Section 7) and the USB/PCIe subsystem (CONFIG_USB_PCI, CONFIG_PCIE_BRCMSTB), none of which are required for a display-less, peripheral-less server deployment. Kernel time reported here is measured via `systemd-analyze time` under a clean production `cmdline.txt` (no `initcall_debug/printk.time` instrumentation), reflecting real-world boot time rather than the instrumented `initcall`-level figures discussed in Section 5.

Subsystem	Removed/Deferred	Time Saved (ms)	Notes
Stock Raspberry Pi OS kernel	— (reference)	—	Baseline: 2000 ms kernel time
Tracing (FTRACE / KPROBE_EVENTS / EVENT_TRACING)	Removed	950	Common to Kernel A and B (Section 5)
DRM/VC4 + VCHIQ (display, audio, camera, codec, ISP)	Removed	10	Measured as one cumulative stage; per-subsystem isolation not separately re-tested under clean <code>cmdline</code>
USB / PCIe (<code>xhci_pci</code> , <code>brcm_pcie</code>)	Removed	230	
Cumulative (stock → final)		1190	2000 ms → 810 ms

Table 2: Kernel A (headless) subsystem removal impact, real-world kernel time.

Note: the DRM/VC4 and VCHIQ removals were built and measured as a single cumulative stage rather than isolated individually under this clean-`cmdline` measurement; the comparatively small 10 ms figure at this stage reflects real-world kernel time and is not directly comparable to the larger reductions observed in the instrumented `initcall`-level data in Section 5, which carries additional logging overhead.

Reproducibility: Configuration Commands per Stage

For reproducibility, the exact `scripts/config` commands applied at each stage, starting from `bcm2711_defconfig`, are given below. Each stage builds on the previous one's already-modified `.config`, followed by `make olddefconfig` to resolve dependent options.

Tracing subsystem (common baseline, Section 5.2)

```
scripts/config --disable CONFIG_FTRACE
scripts/config --disable CONFIG_KPROBE_EVENTS
scripts/config --disable CONFIG_EVENT_TRACING
scripts/config --disable CONFIG_KPROBES
make olddefconfig
```

DRM/VC4 and VCHIQ removal (Stage 2)

```

scripts/config --disable CONFIG_DRM_VC4
scripts/config --disable CONFIG_BCM2835_VCHIQ
scripts/config --disable CONFIG_BCM2835_VCHIQ_MMAL
scripts/config --disable CONFIG_BCM_VC_SM_CMA
scripts/config --disable CONFIG_SND_BCM2835
scripts/config --disable CONFIG_VIDEO_BCM2835
scripts/config --disable CONFIG_VIDEO_BCM2835_UNICAM
scripts/config --disable CONFIG_VIDEO_BCM2835_UNICAM_LEGACY
scripts/config --disable CONFIG_VIDEO_CODEC_BCM2835
scripts/config --disable CONFIG_VIDEO_ISP_BCM2835
make olddefconfig

```

USB/PCIe removal (Stage 3)

```

scripts/config --disable CONFIG_USB_PCI
scripts/config --disable CONFIG_PCIE_BRCMSTB
make olddefconfig

```

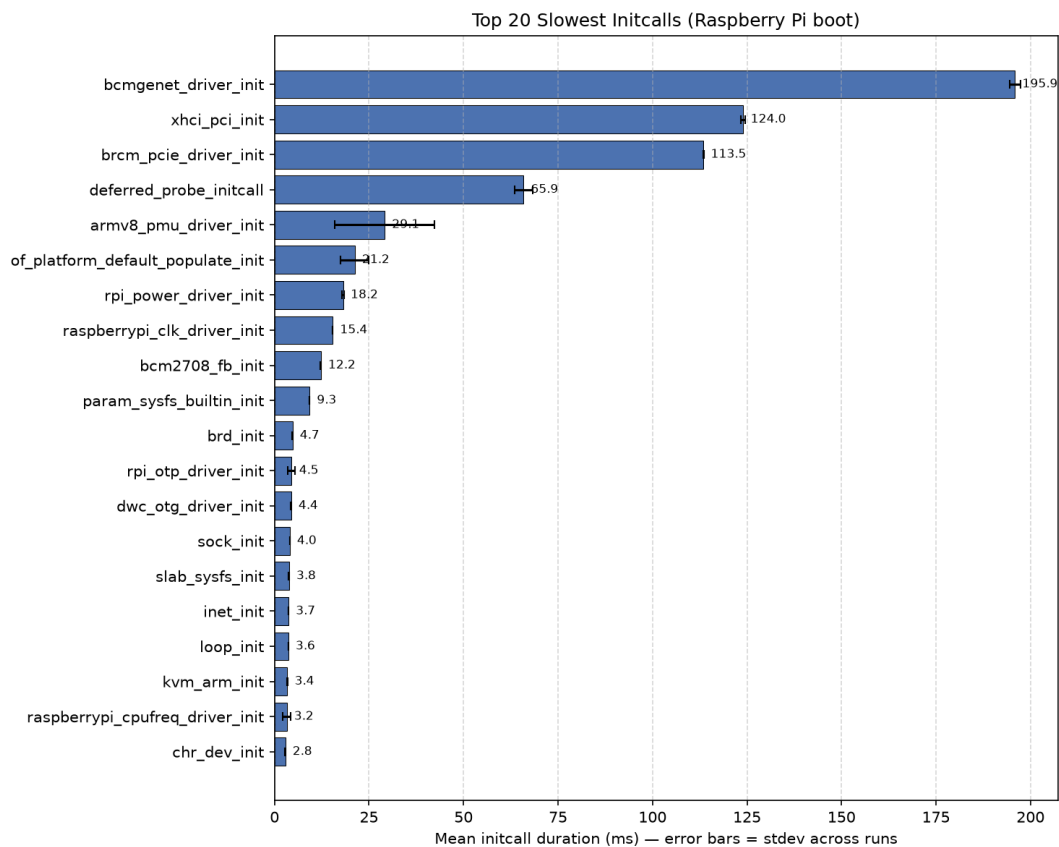


Figure 3: Kernel initcall timing after DRM/VC4 and VCHIQ removal (Stage 2).

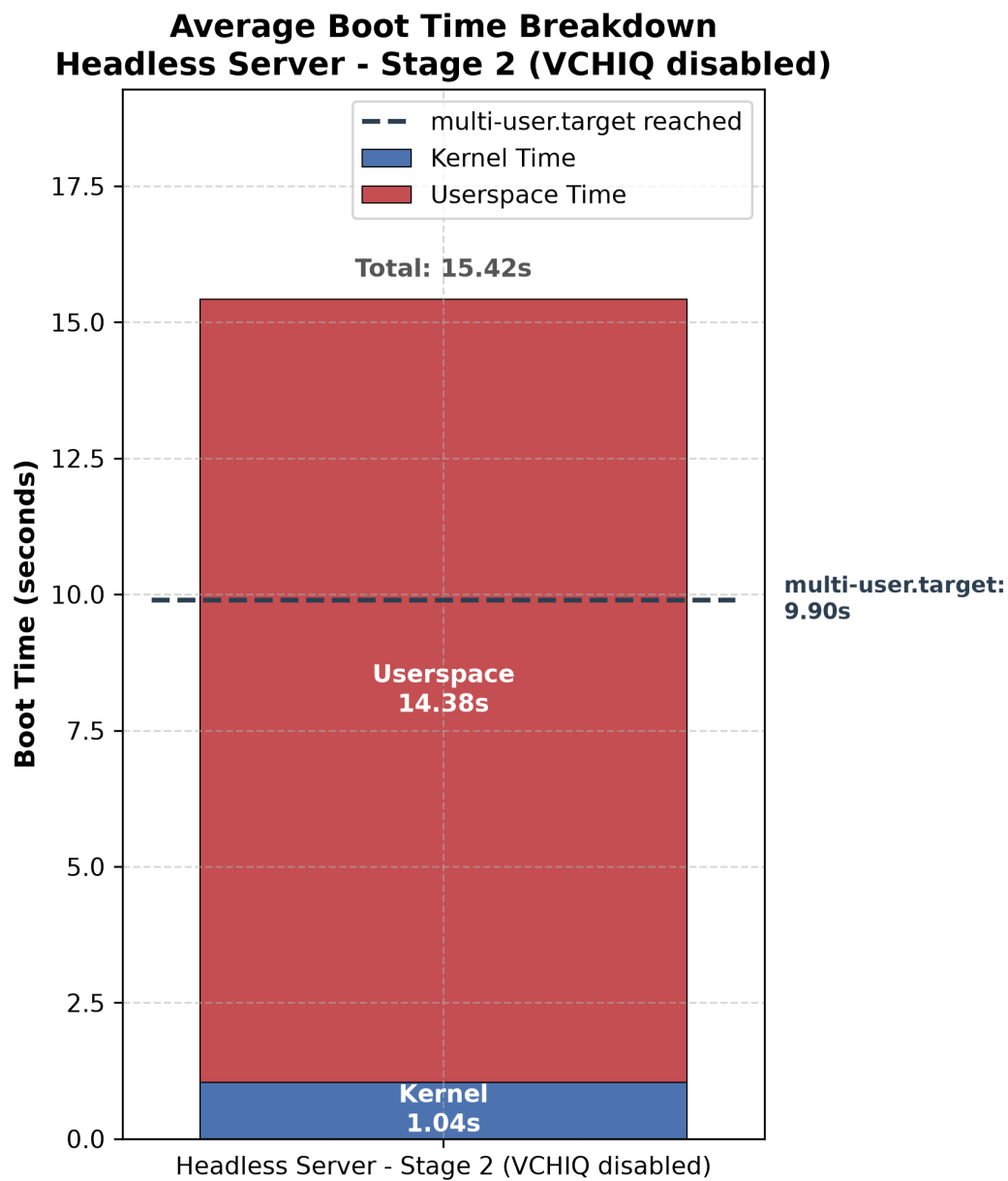


Figure 4: Total boot time after DRM/VC4 and VCHIQ removal (Stage 2).

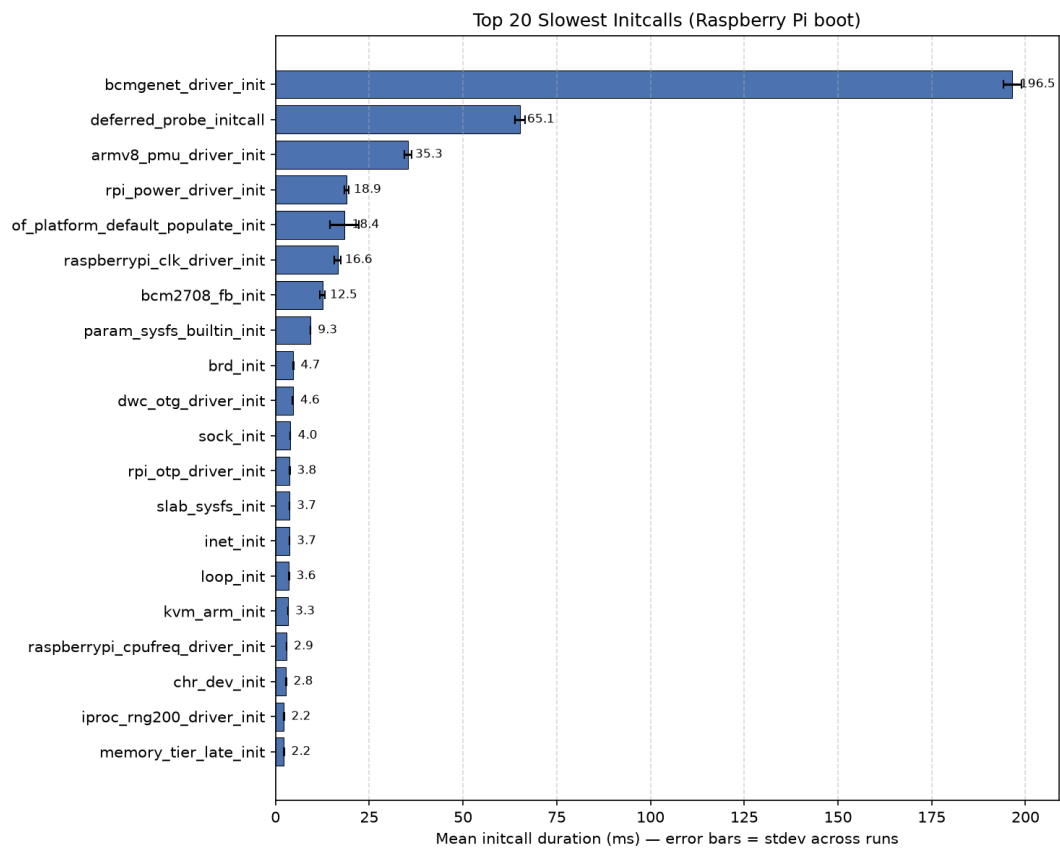


Figure 5: Kernel initcall timing after USB/PCIe removal (Stage 3, final headless kernel).

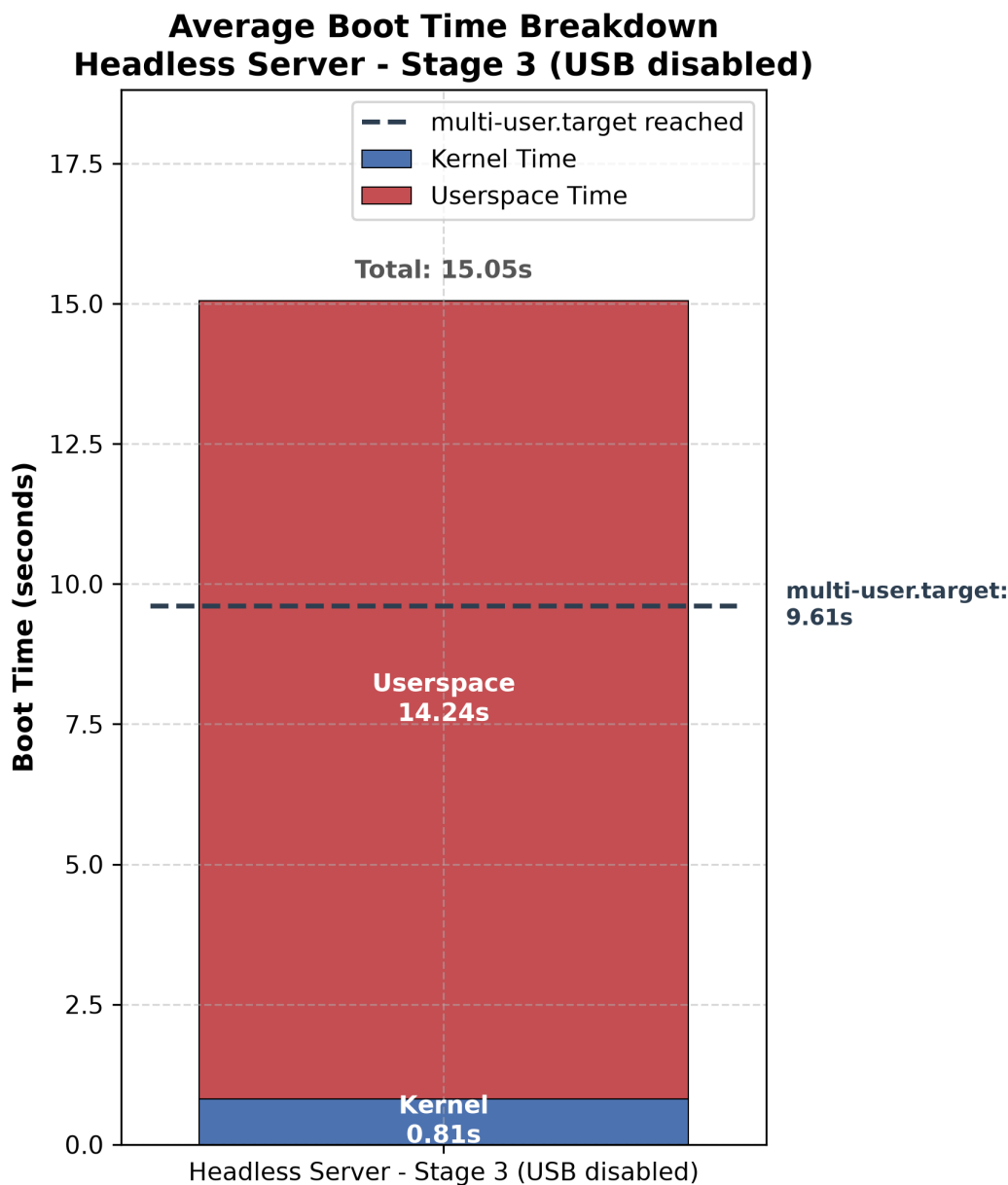


Figure 6: Total boot time after USB/PCIe removal (Stage 3, final headless kernel).

7.2 Kernel B: Interactive Unit

For the Interactive Unit profile, the VideoCore display stack (CONFIG_DRM_VC4), VCHIQ and its audio/camera dependents, USB support, and Bluetooth are retained as functional requirements. A Raspberry Pi OS (with Desktop) deployment requires HDMI output, an audio path, USB input, and Bluetooth support. Consequently, the Headless-specific removals in Section 7.1 are not applied here. Kernel-level optimization for this profile is limited to the tracing-subsystem removal common to both deployment paths.

Kernel Version Consistency: the .34/.39 Discovery

During setup of the Interactive Unit, the Raspberry Pi OS (with Desktop) image initially ran kernel 6.18.34+rpt-rpi-v8. After apt update/upgrade, the system moved to 6.18.39+rpt-rpi-v8. The populated images/interactive-mode/misc-kernel-after-update/

figures document this post-upgrade state.

Under the .39 kernel, `gpio_led_driver_init` was no longer present in the measured initcall profile. This contrasts with the earlier .34-based capture, where the initcall was visible. The observation therefore establishes an association between the initcall's presence and the kernel point version; the precise upstream cause was not established within this study.

A full configuration comparison found no meaningful functional driver or subsystem difference in the relevant configurations. The .39 capture is therefore treated as empirical evidence of point-version-dependent behavior, not as an additional optimization stage.

For a valid comparison between the Headless and Interactive Unit profiles, the final benchmark configuration uses the same controlled kernel version, `6.18.34+rpt-rpi-v8`, for both profiles. Automatic kernel updates are prevented with:

```
sudo apt-mark hold raspberrypi-kernel raspberrypi-bootloader
sudo systemctl disable --now apt-daily-upgrade.timer apt-daily.timer
```

Both benchmark profiles are therefore based on kernel version `6.18.34+rpt-rpi-v8`. The `6.18.39` result remains in the report as a documented version-dependence observation rather than a benchmark stage.

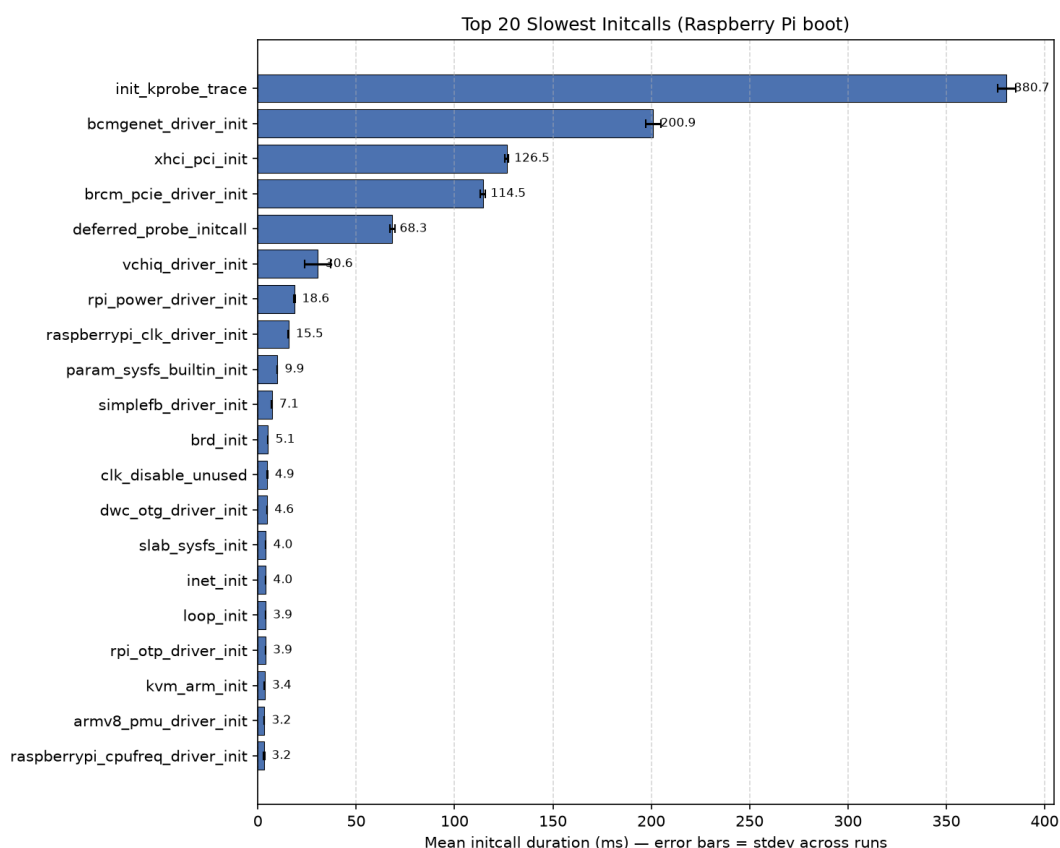


Figure 7: Interactive Unit after upgrade to kernel `6.18.39+rpt-rpi-v8`; `gpio_led_driver_init` is absent from the measured initcall profile.

8 Userspace Optimization

The userspace investigation is split into two deployment-specific paths. The Headless Server path prioritizes minimum boot latency and a minimal service footprint, while the Interactive Unit retains desktop-oriented services and NetworkManager for graphical usability and ease of configuration.

8.1 Headless Server

The Headless Server userspace track begins from the fully kernel-optimized Stage 3 image with default userspace configuration.

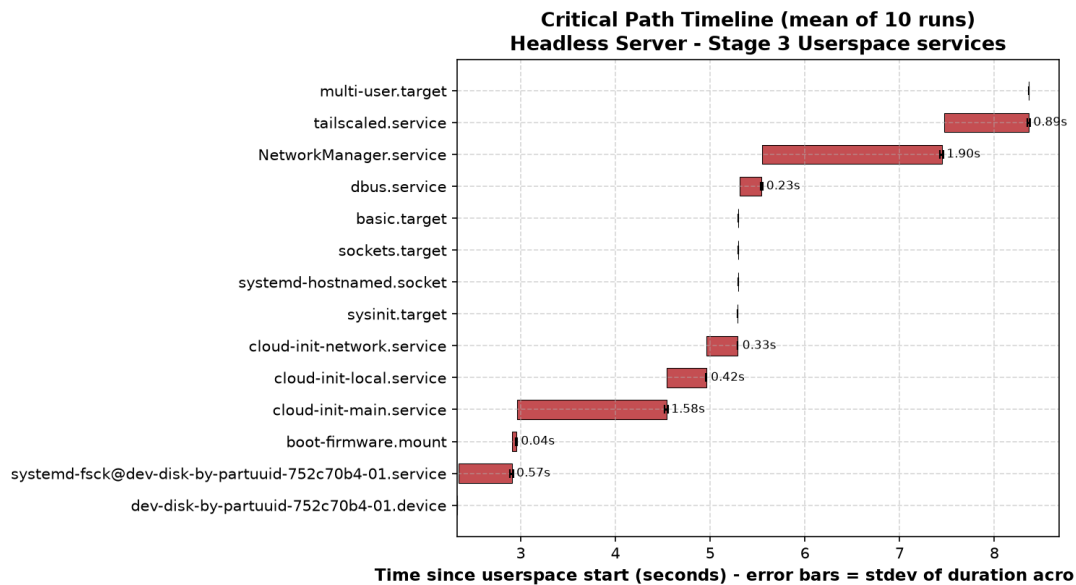


Figure 8: Critical path timeline, default Headless Server userspace configuration.

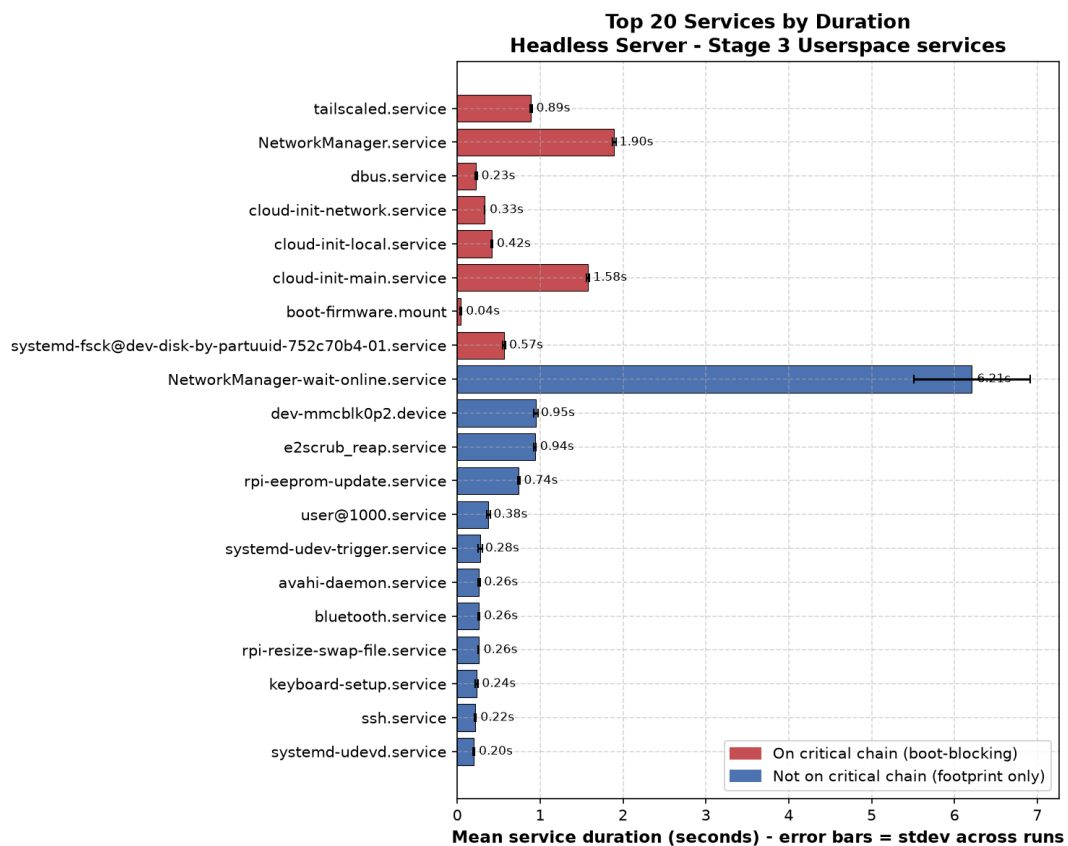


Figure 9: Top services by duration, default Headless Server userspace configuration.

Service	Mean Time (ms)	Removable?	Purpose
cloud-init-main.service	1578.4	Yes	First-boot provisioning only
NetworkManager.service	1895.3	No	Managed Wi-Fi/network
tailscaled.service	892.2	No	Remote SSH access
systemd-fsck@....service	565.1	No	Filesystem integrity check
cloud-init-local.service	416.4	Yes	First-boot provisioning only
cloud-init-network.service	329.6	Yes	First-boot provisioning only
dbus.service	229.5	No	Core IPC bus
boot-firmware.mount	43.6	No	Boot partition mount

Table 3: Principal userspace services in the default Headless Server configuration.

Cloud-init Removal

Since first-boot provisioning is complete on the test device, the `cloud-init` service group was disabled.

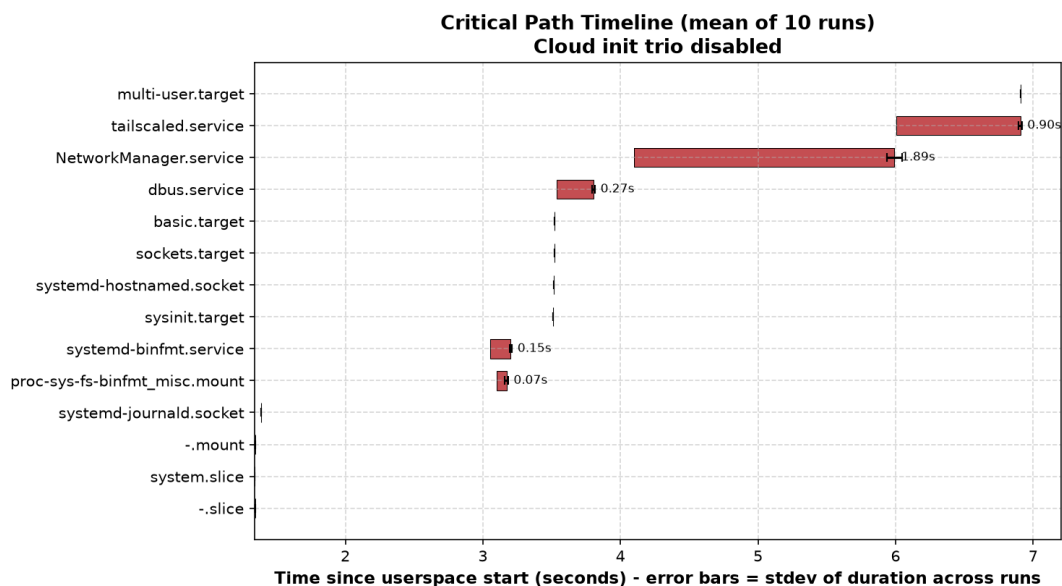


Figure 10: Critical path after disabling the cloud-init service group.

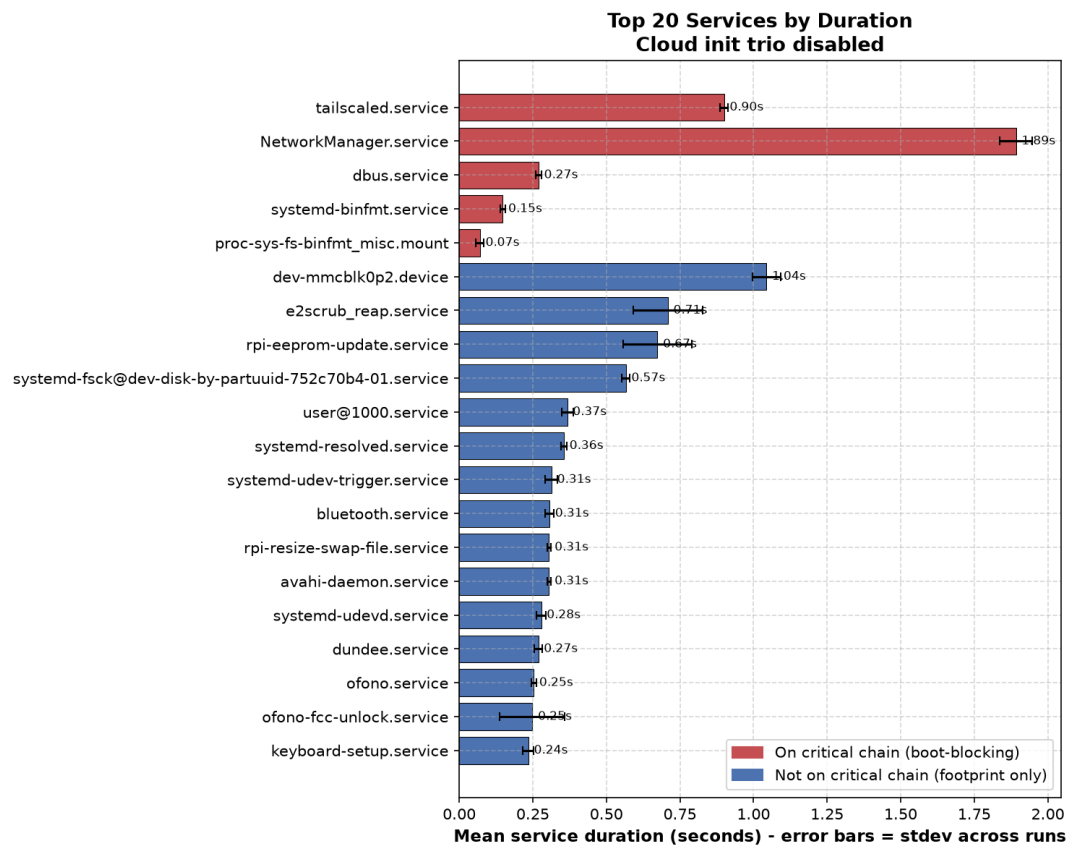


Figure 11: Top services after disabling cloud-init.

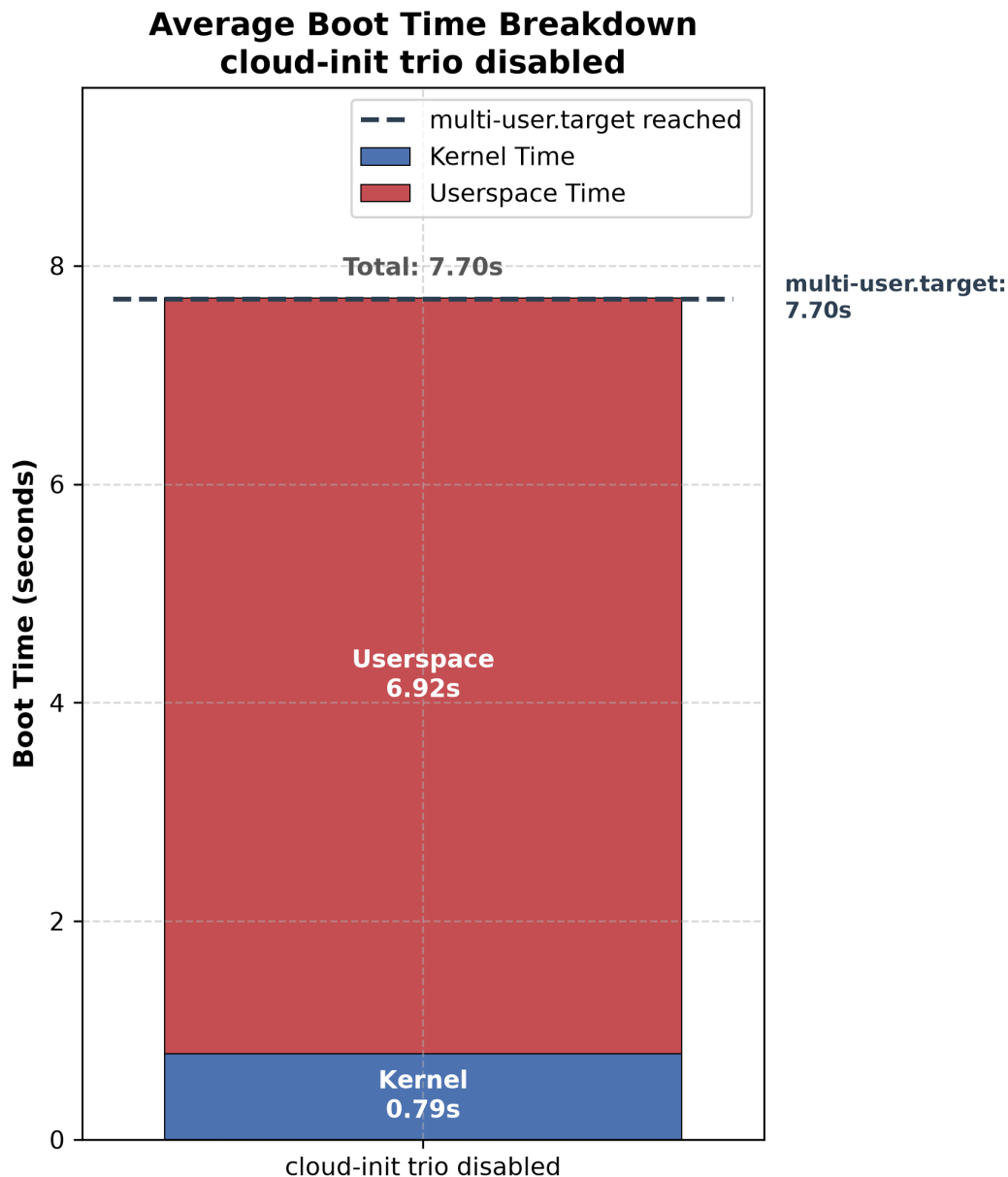


Figure 12: Total boot time after disabling cloud-init.

Network Management Stack: Migration to systemd-networkd + wpa_supplicant

NetworkManager was replaced with `systemd-networkd` paired with `wpa_supplicant`. The unused Ethernet interface was explicitly excluded from management from the outset, avoiding the carrier-wait artifact by construction.

```
# /etc/systemd/network/10-eth0.network
[Match]
Name=eth0

[Link]
Unmanaged=yes

# /etc/systemd/network/20-wlan0.network
[Match]
Name=wlan0
```

```
[Network]
DHCP=yes
```

ConnMan was evaluated only during an earlier troubleshooting stage. It is not part of the final comparison and is retained only as a side note because the earlier evaluation exposed DNS/Tailscale interaction issues that made it unsuitable for the final headless configuration.

The final `systemd-networkd + wpa_supplicant` configuration used Tailscale's default service ordering without an additional wait-online dependency. BSSID/channel pinning was tested but not retained because its measured benefit was negligible and it reduces roaming/failover flexibility.

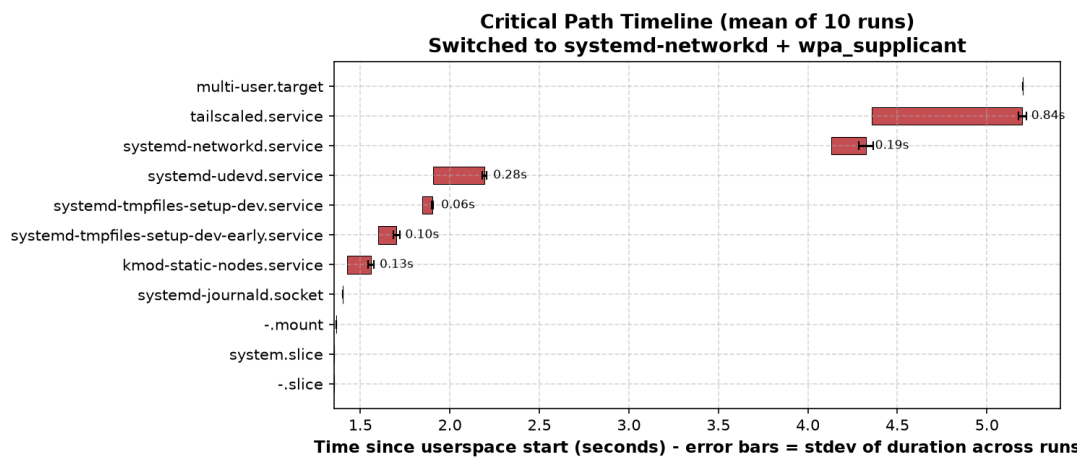


Figure 13: Critical path for the final `systemd-networkd + wpa_supplicant` configuration.

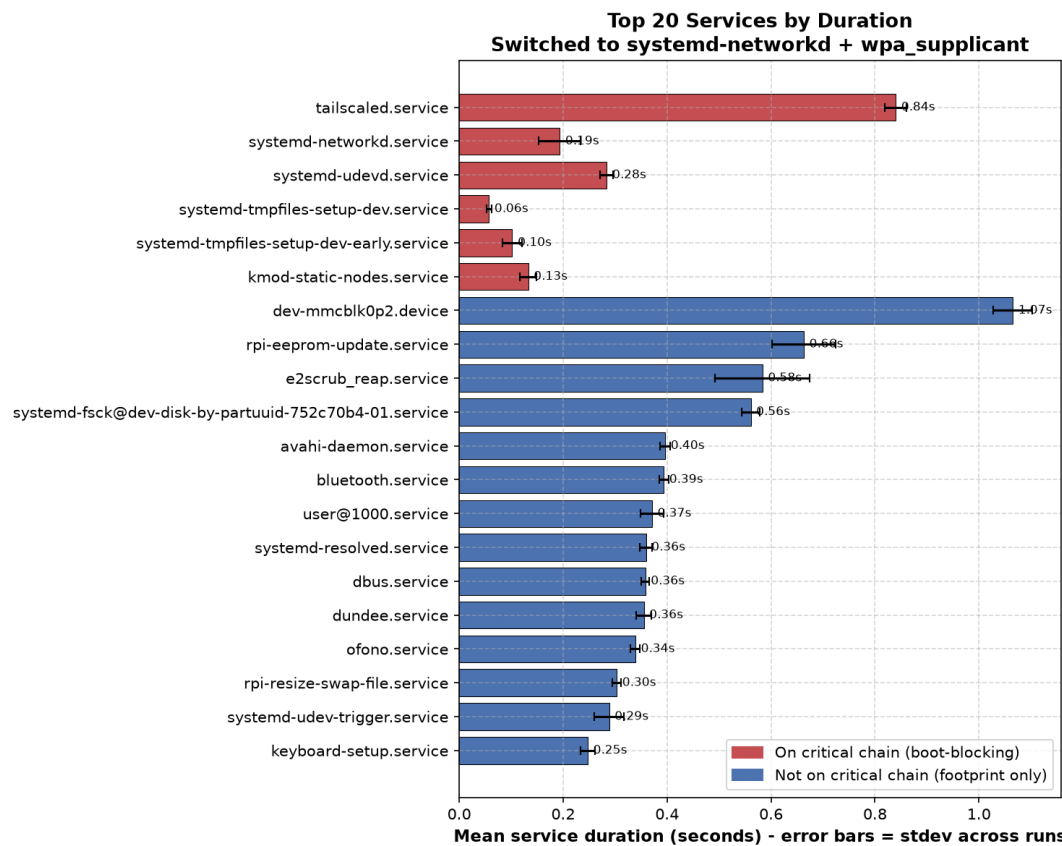


Figure 14: Top services for the final systemd-networkd + wpa_supplicant configuration.

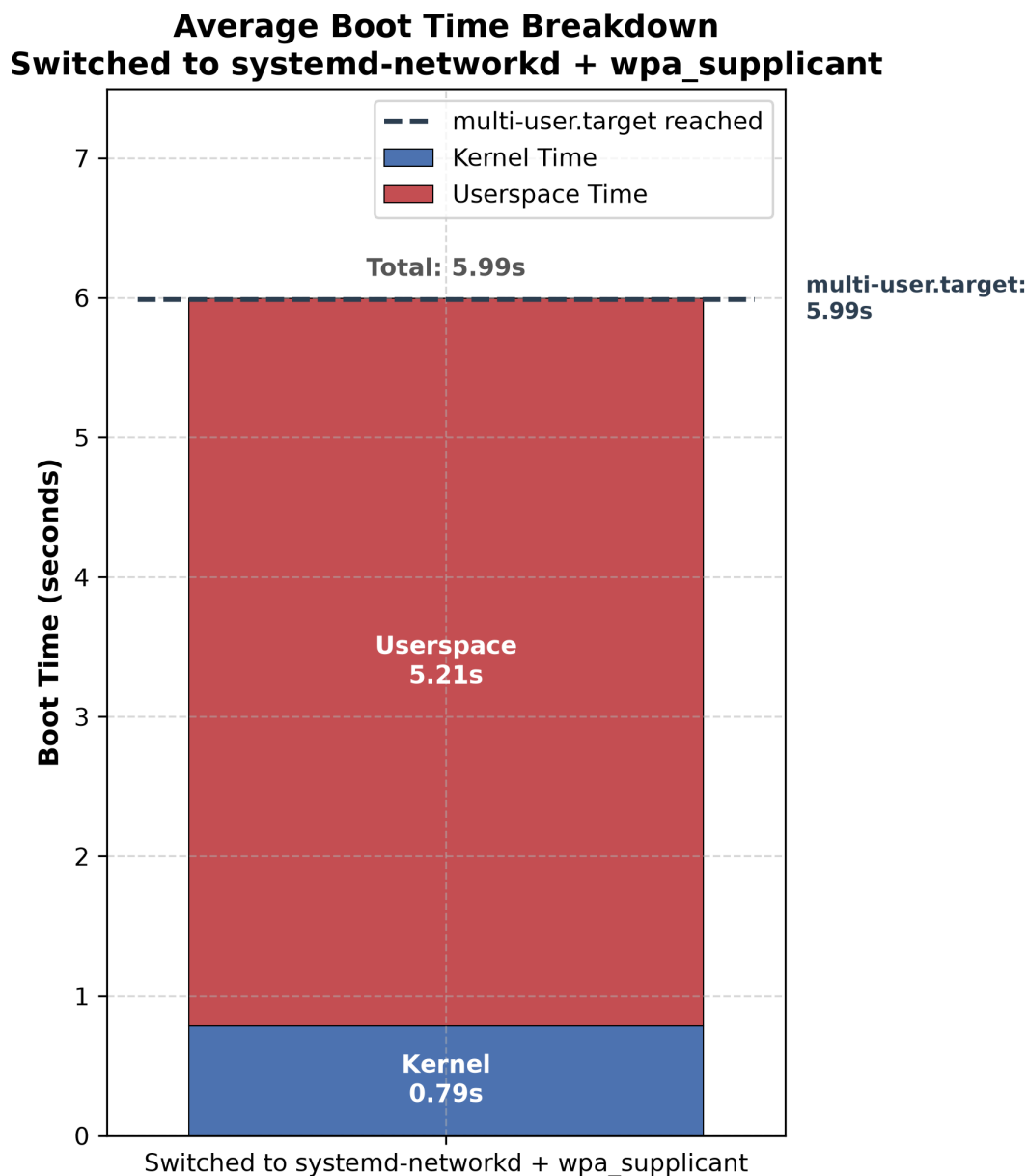


Figure 15: Total boot time for the final systemd-networkd + wpa_supplicant configuration.

The corrected network-stack investigation showed that the previously reported multi-second WPA-Enterprise cost was confounded by the permanently disconnected `eth0` interface. Genuine EAP/PEAP authentication was independently measured at under 10 ms. The final network-stack configuration reduced total boot time to approximately 6.0s before optional-service cleanup.

Optional / Off-Critical-Chain Services

Bluetooth, avahi, and keyboard-setup were disabled because they are not required by the Headless Server profile.

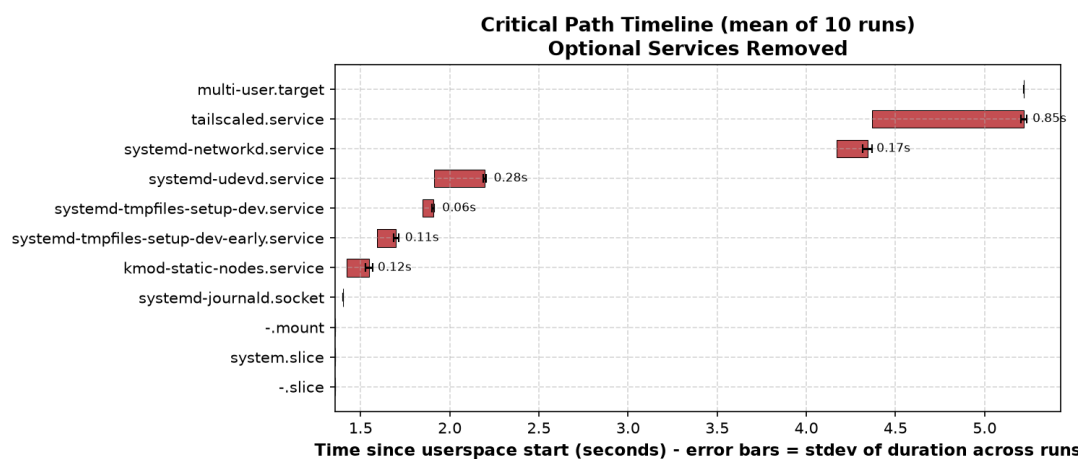


Figure 16: Critical path after removal of optional services.

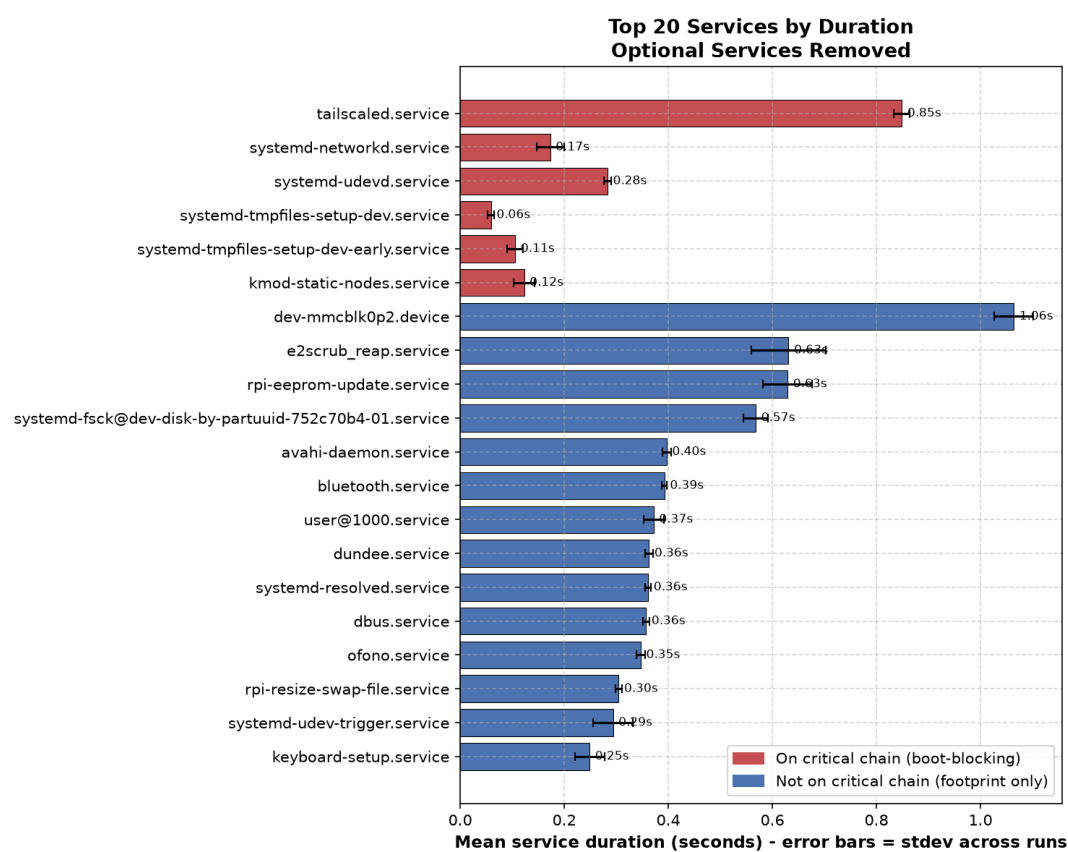


Figure 17: Top services after removal of optional services.

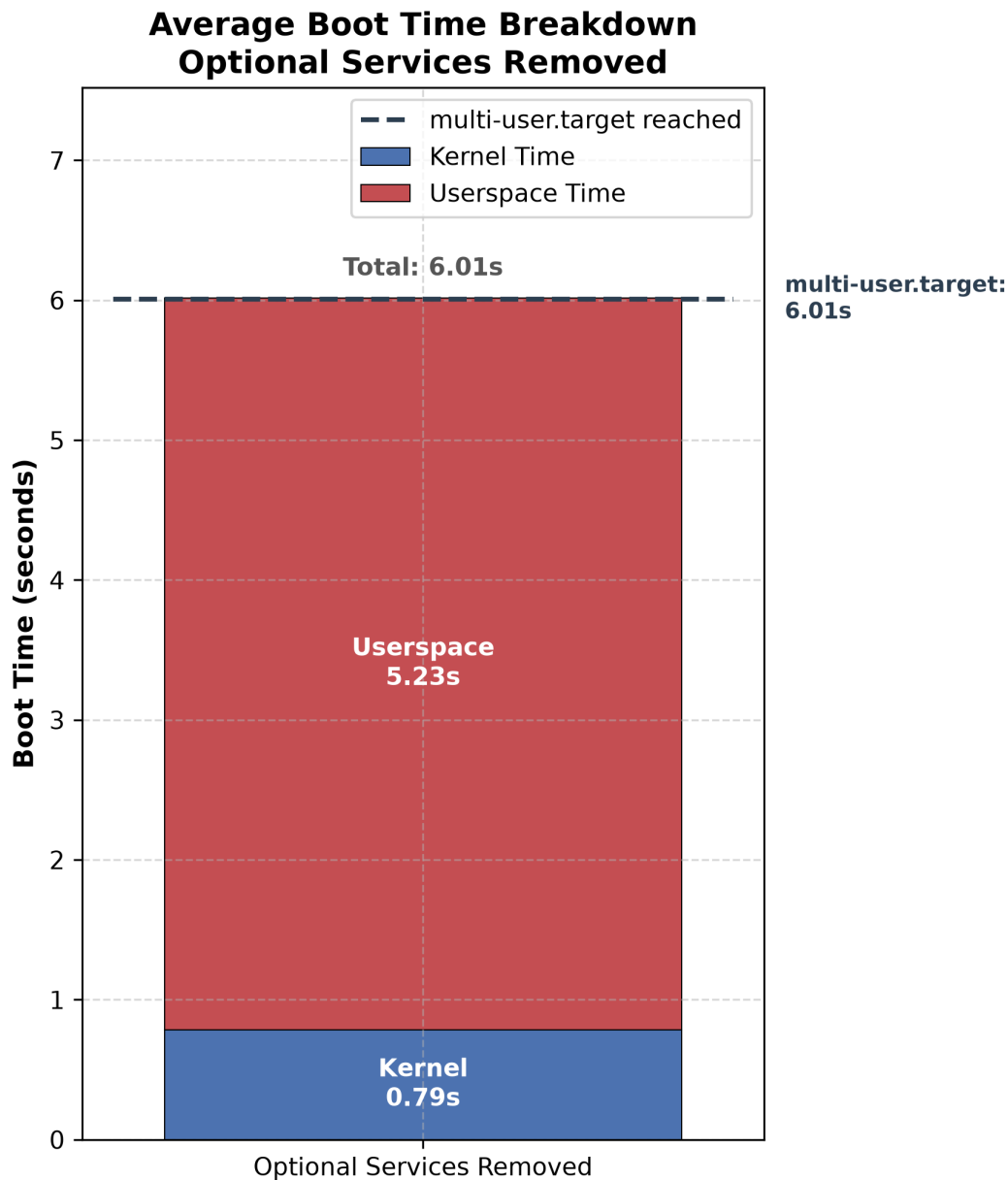


Figure 18: Final Headless Server boot time after optional-service cleanup.

The final Headless Server configuration combines the optimized kernel with cloud-init removal, `systemd-networkd` + `wpa_supplicant`, the final Tailscale ordering, and removal of unused optional services. Total boot time is approximately **6.01 s** (kernel 0.79 s + userspace 5.23 s).

8.2 Interactive Mode

The Interactive Unit retains the desktop functionality required by the profile. NetworkManager is retained because its GUI integration and ease of configuration are appropriate for an interactive deployment. The unused Ethernet interface is nevertheless excluded from NetworkManager management so that its disconnected carrier state does not introduce unnecessary background work.

Baseline Userspace Profile

The populated baseline measurements include service ranking, critical-chain timing, total boot milestones, and kernel initcall profiling.

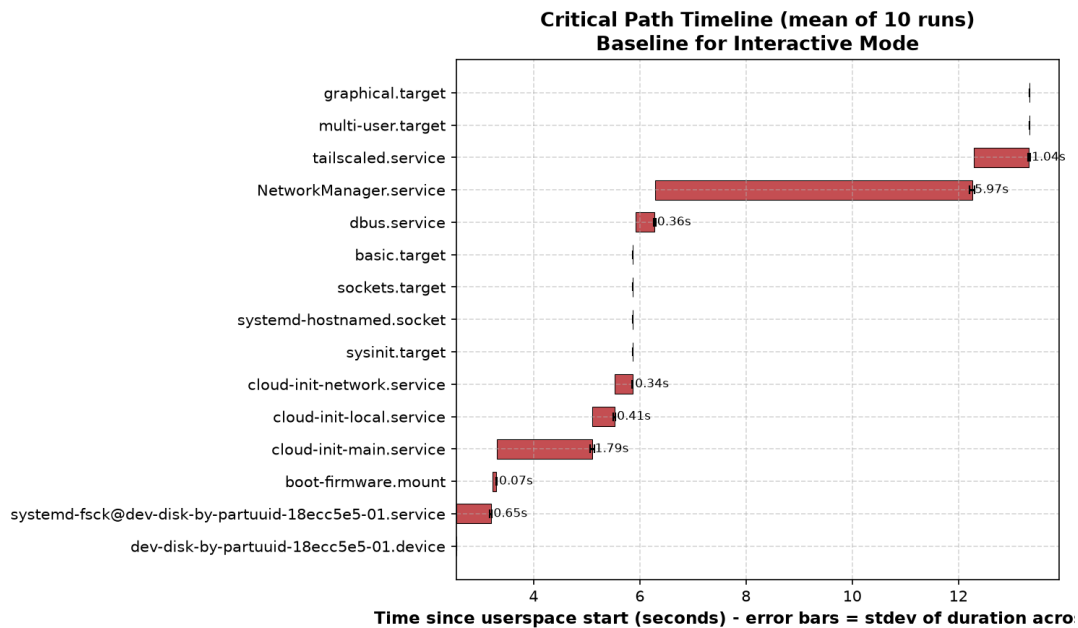


Figure 19: Interactive Unit baseline critical-chain timeline.

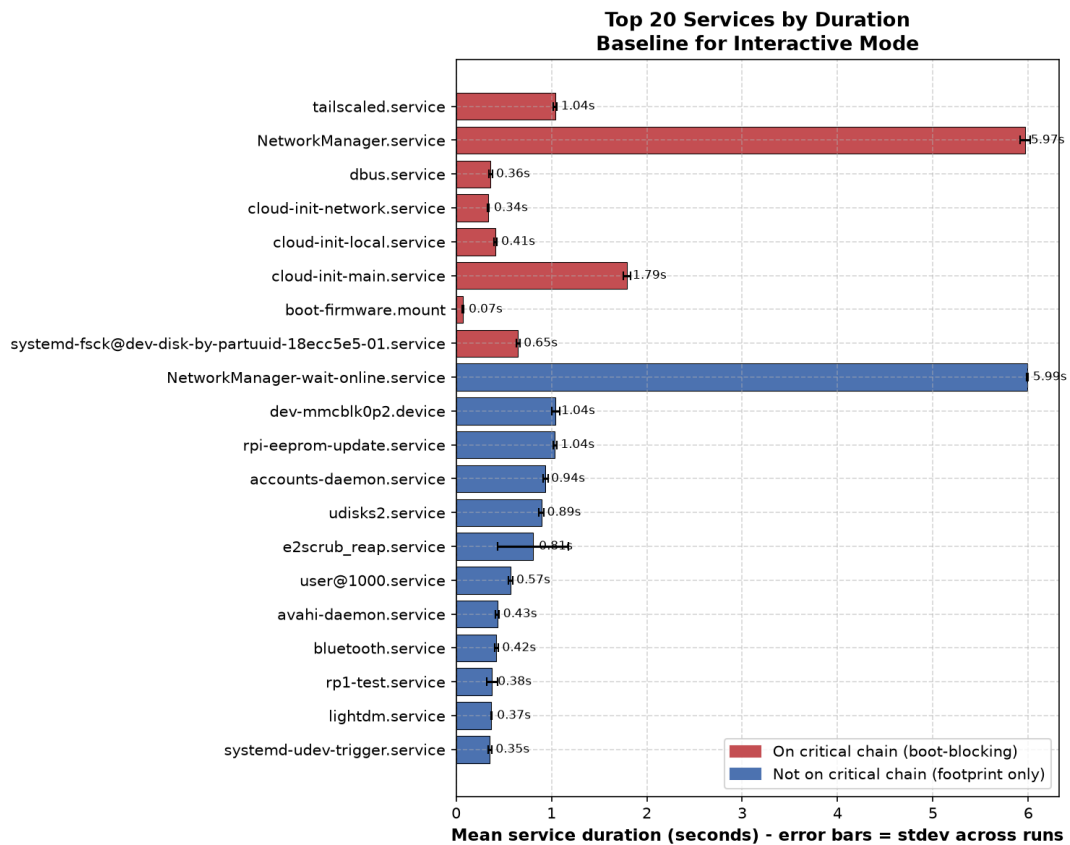


Figure 20: Interactive Unit baseline service-duration ranking.

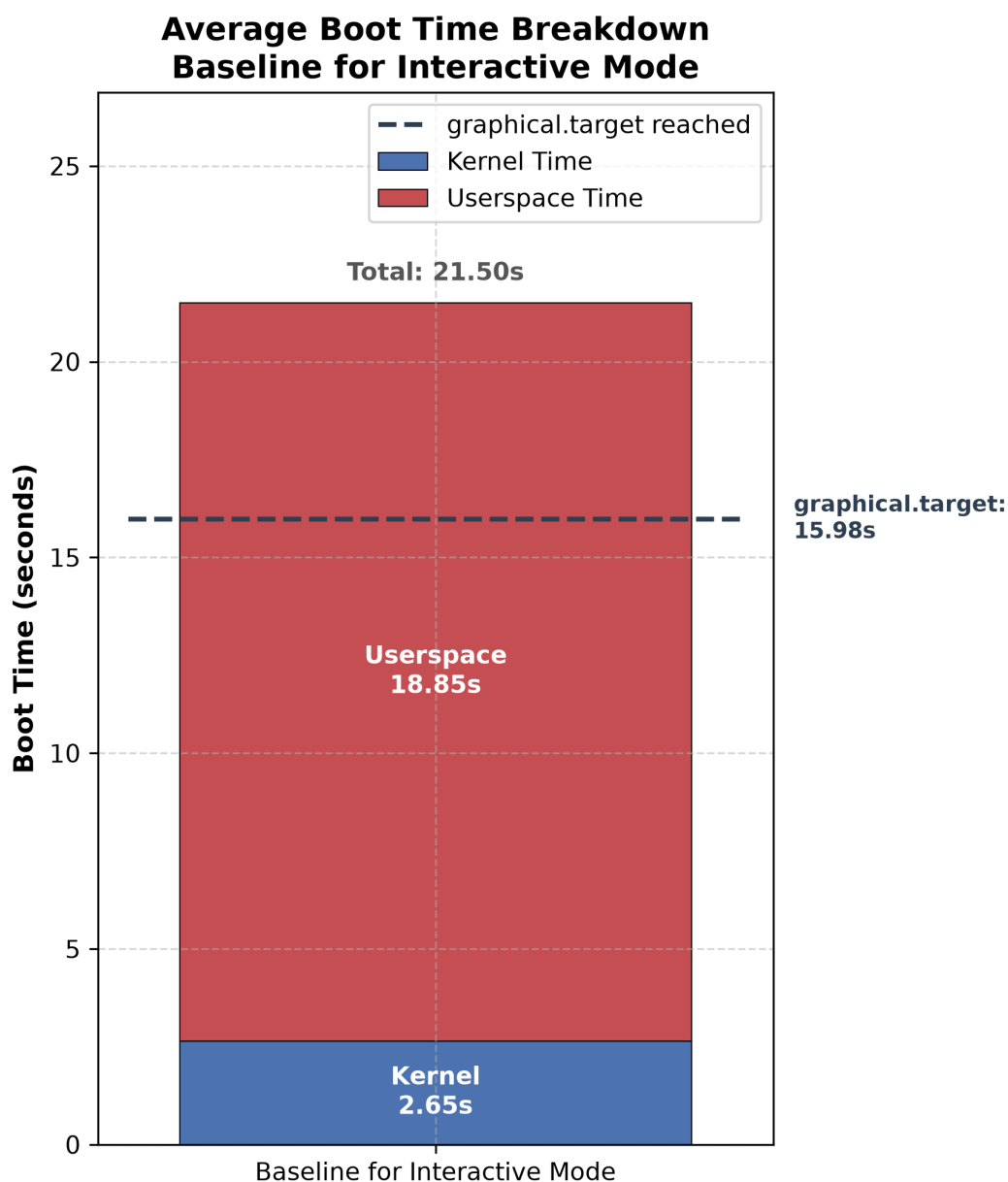


Figure 21: Interactive Unit baseline boot milestones.

Tracing-Disabled Kernel and Cloud-init Removal

The Interactive Unit kernel retains display, audio, USB, and Bluetooth functionality; the common tracing optimization is retained. Its userspace path then removes the completed first-boot cloud-init provisioning workload.

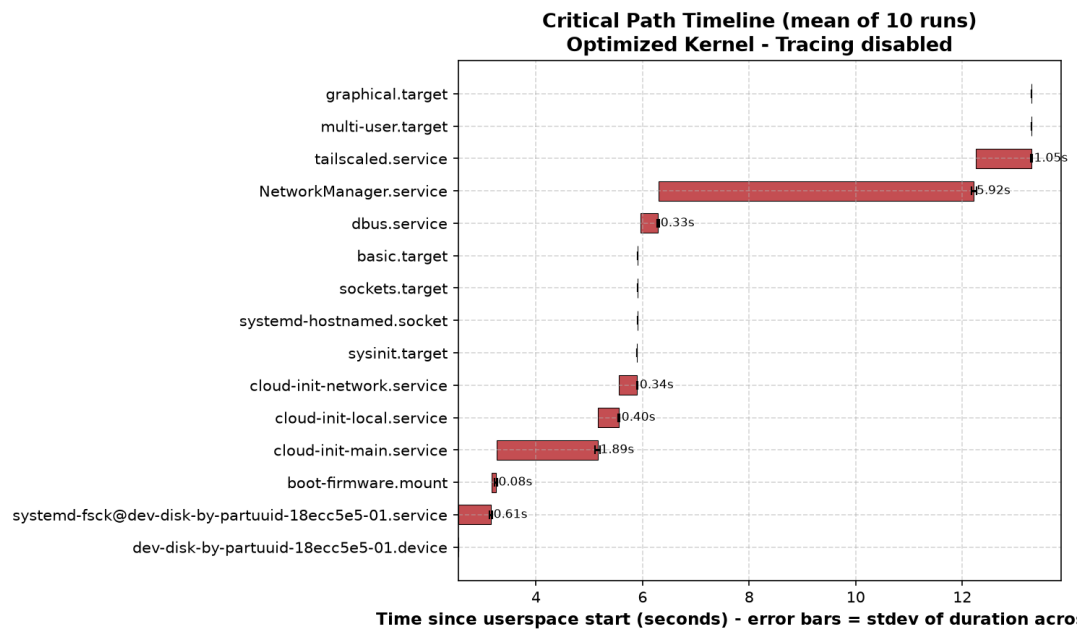


Figure 22: Interactive Unit after tracing-subsystem removal.

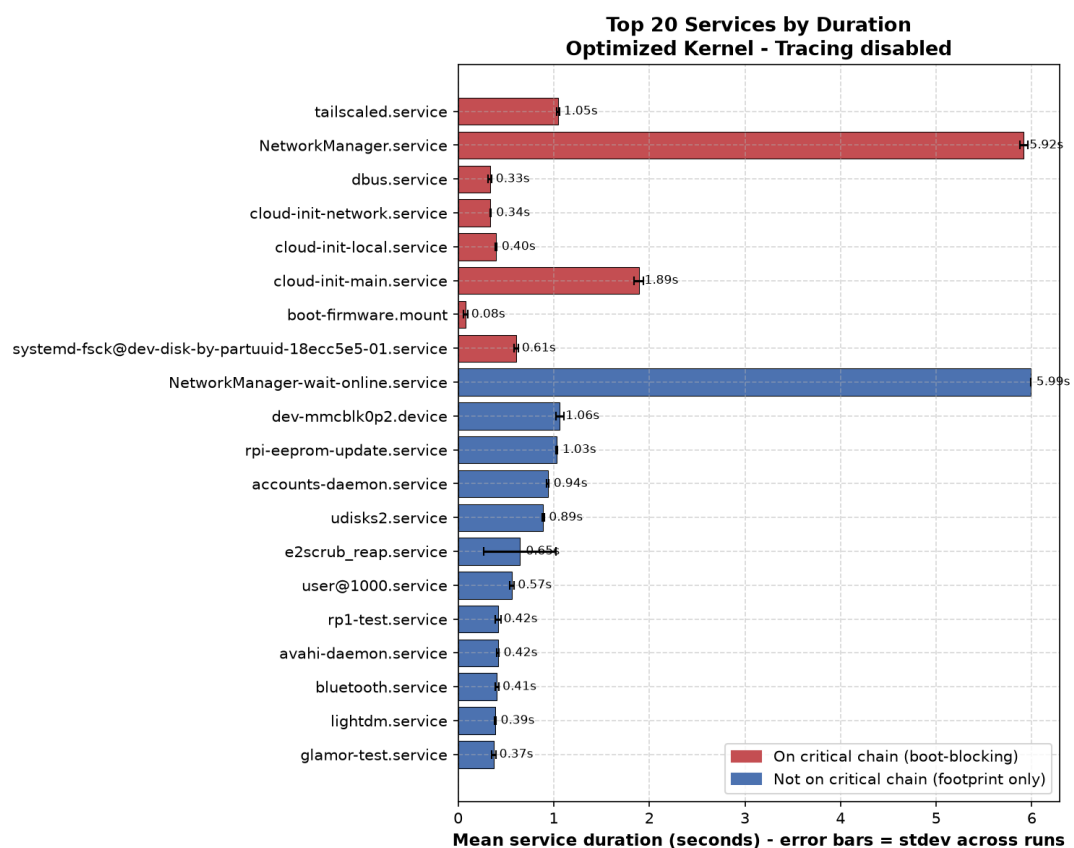


Figure 23: Interactive Unit service ranking after tracing-subsystem removal.

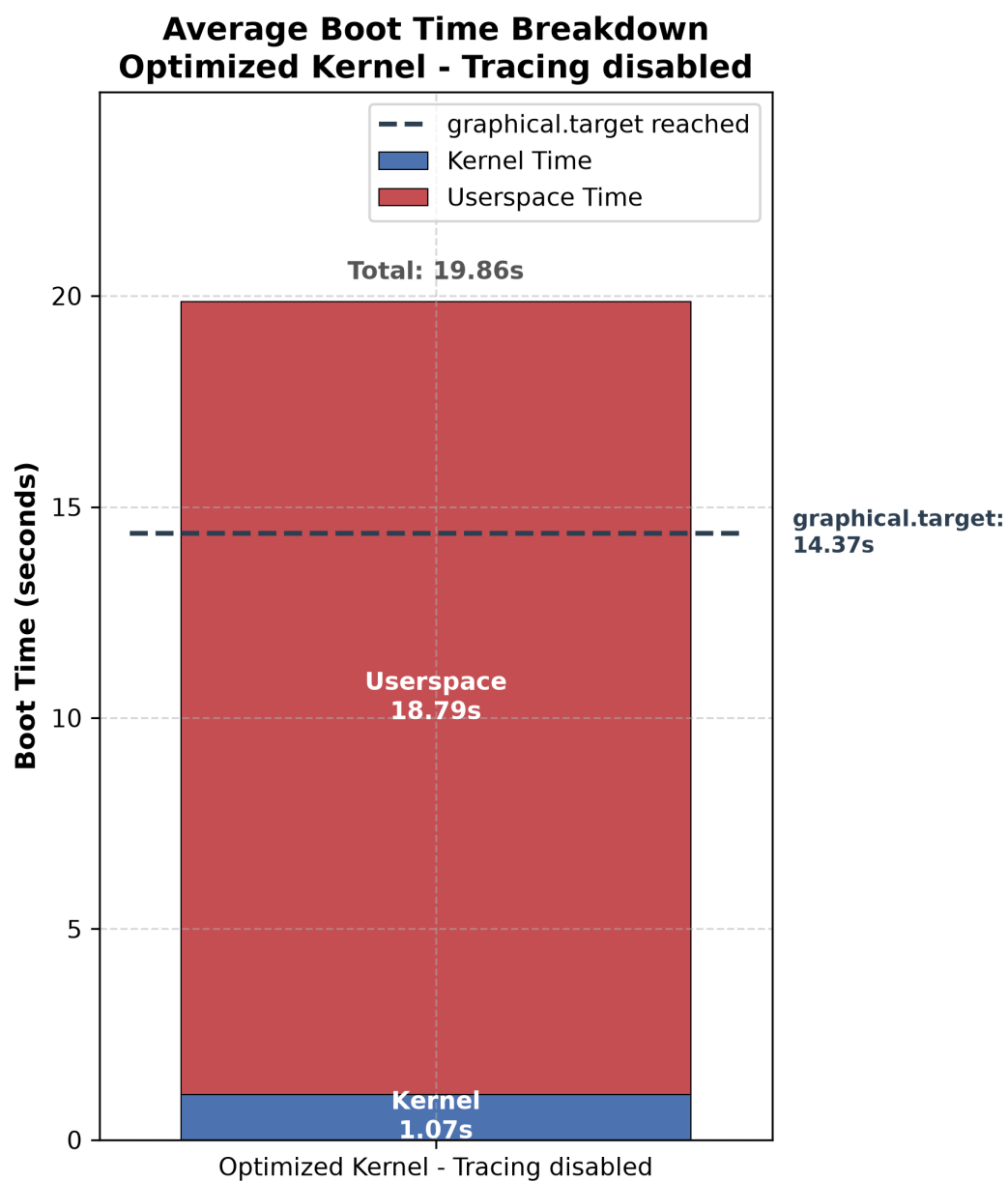


Figure 24: Interactive Unit boot milestones after tracing-subsystem removal.

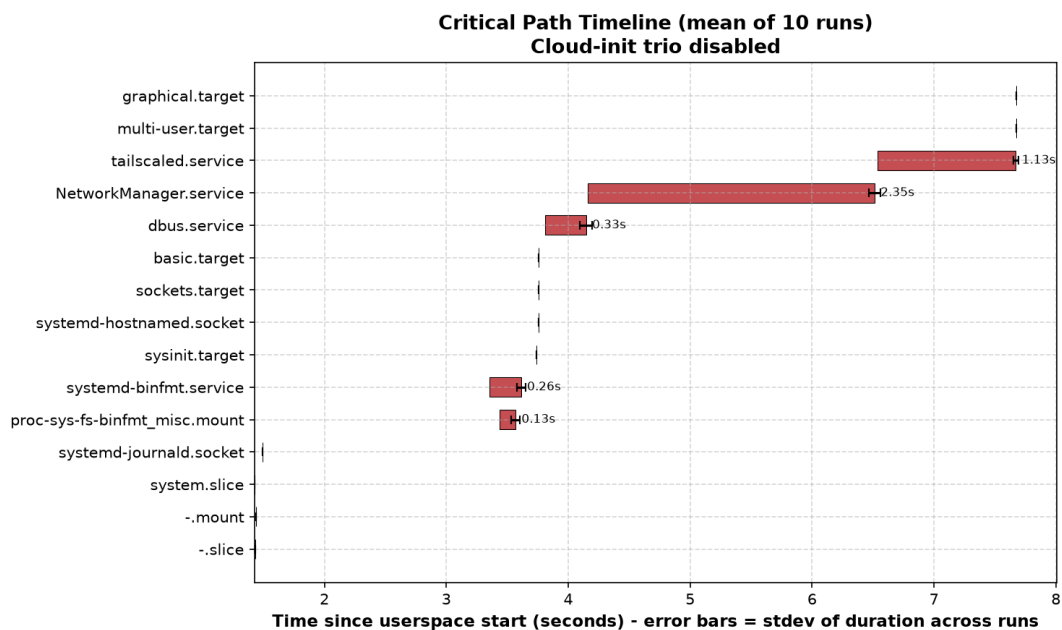


Figure 25: Interactive Unit critical chain after cloud-init removal.

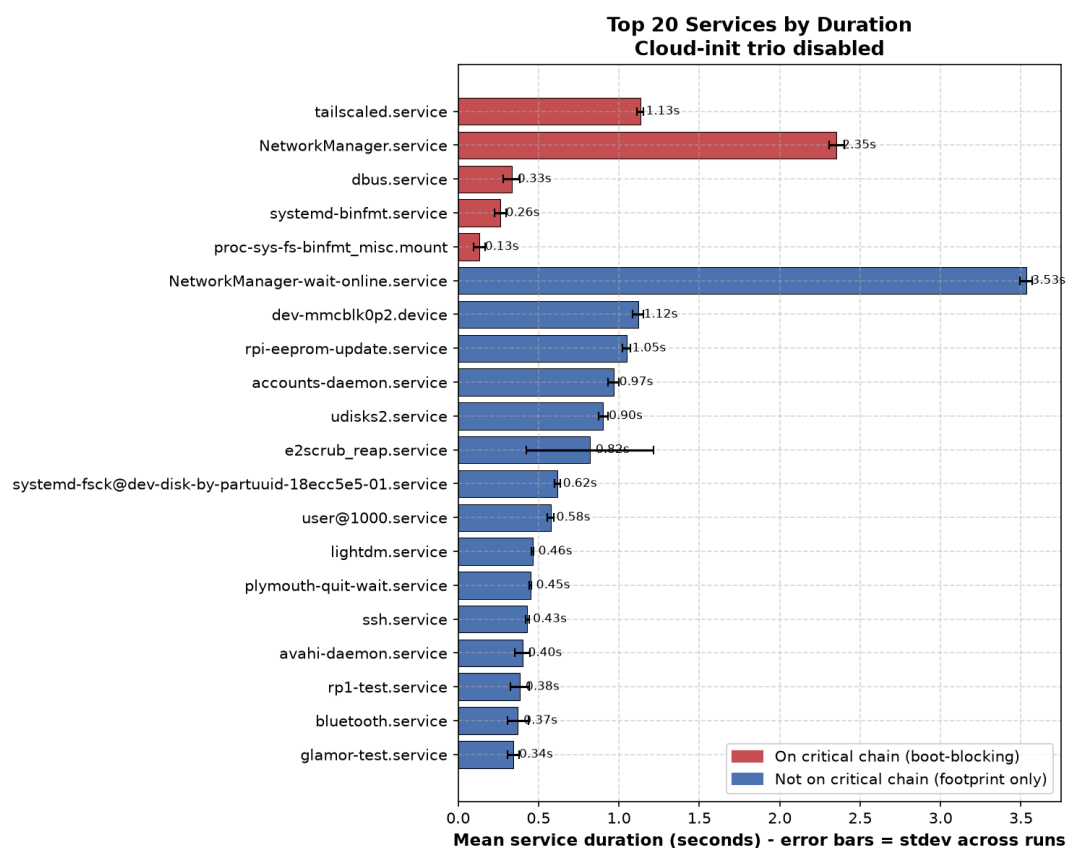


Figure 26: Interactive Unit service ranking after cloud-init removal.

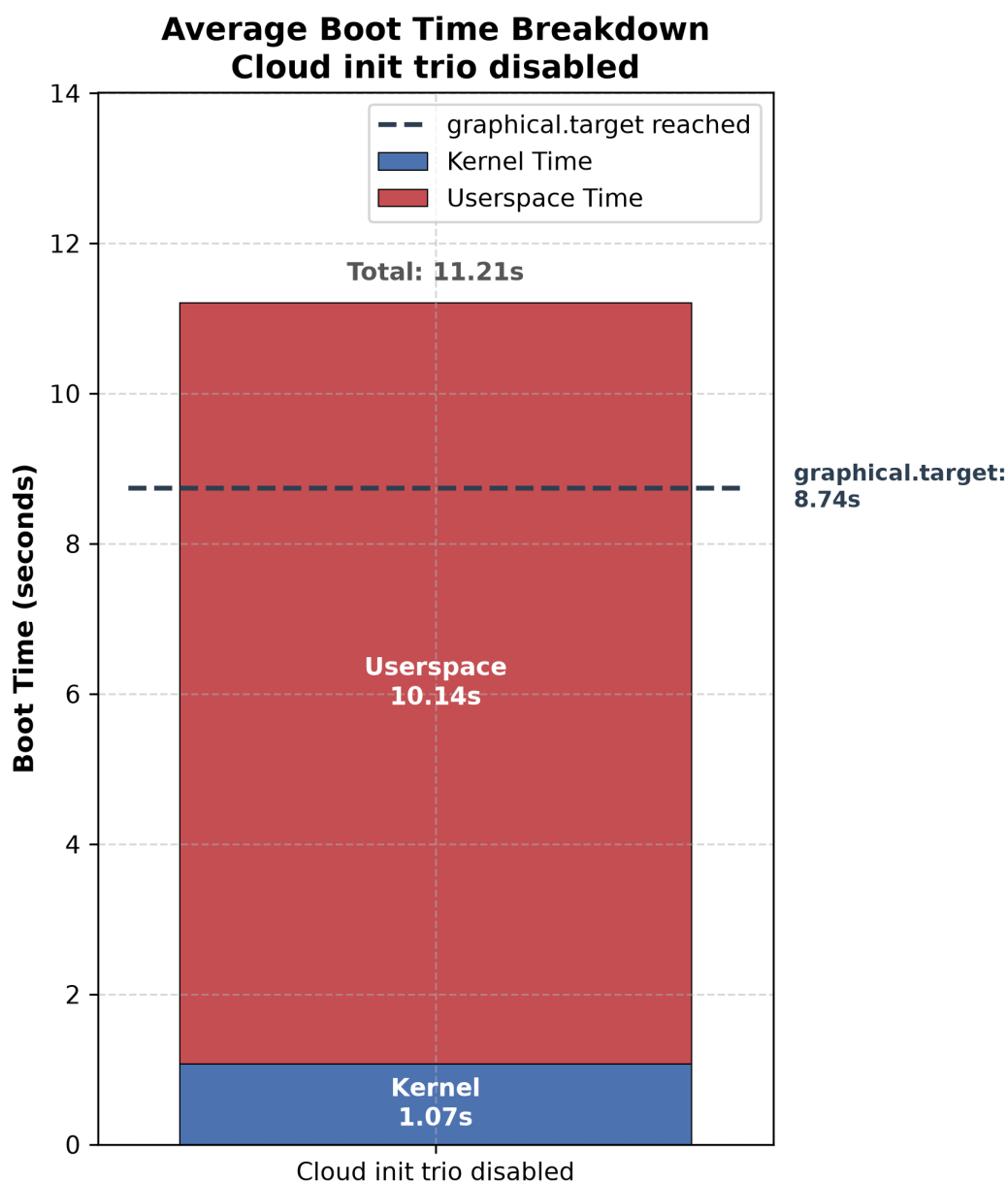


Figure 27: Interactive Unit boot milestones after cloud-init removal.

NetworkManager Configuration

NetworkManager is retained as the final Interactive Unit network-management stack. The unused Ethernet interface is excluded using:

```
# /etc/NetworkManager/conf.d/99-unmanage-eth0.conf
[keyfile]
unmanaged-devices=interface-name:eth0
```

This combines the functional convenience of NetworkManager for the graphical deployment with explicit suppression of the known disconnected-eth0 carrier-wait artifact.

9 Final Results

9.1 Headless Server: Optimized Kernel + Userspace

The final Headless Server configuration combines the Stage 3 optimized kernel with the final userspace configuration. Total boot time is:

Configuration	Kernel (s)	Userspace (s)	Total (s)
Stock	~2.0	13.96	15.96
Final optimized	0.79	5.23	6.01

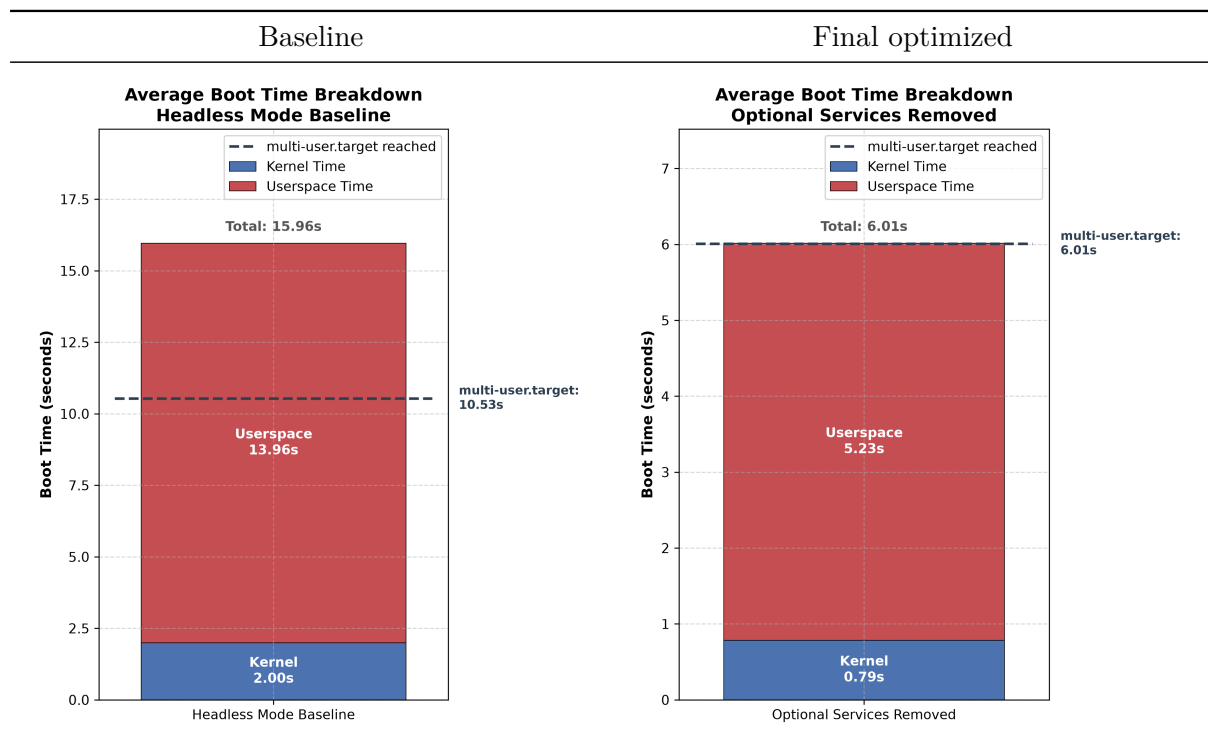


Table 4: Headless Server visual comparison: stock baseline versus fully optimized configuration.

9.2 Interactive Unit: Optimized Kernel + Userspace

The Interactive Unit final populated optimization path consists of the baseline, common tracing-disabled kernel stage, and cloud-init userspace stage. NetworkManager remains the final network-management stack, with the unused Ethernet interface explicitly excluded from management.

Configuration	Kernel (s)	Userspace (s)	Total (s)
Stock	2.65	18.85	21.5
Final optimized	1.07	10.14	11.21

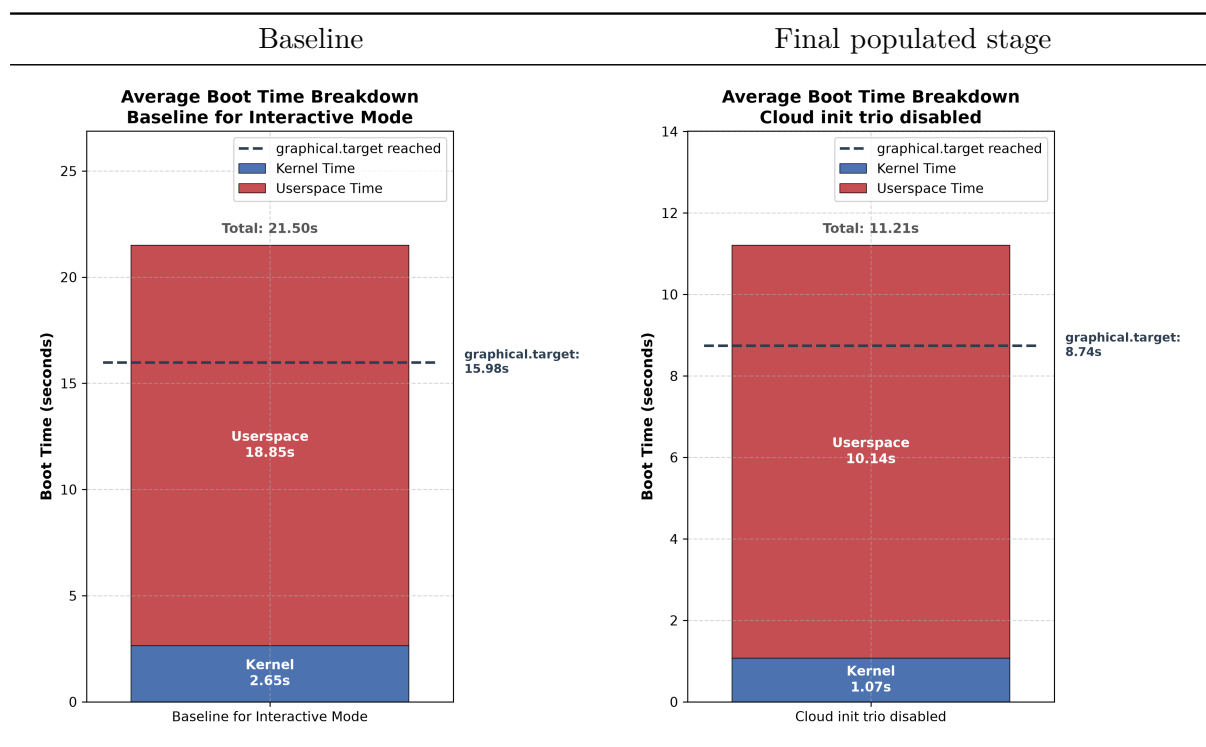


Table 5: Interactive Unit visual comparison: baseline versus final populated userspace optimization stage.

10 Conclusion

This project demonstrates that meaningful, well-documented boot-time reduction is achievable through single-variable-per-stage kernel configuration changes combined with targeted userspace service management, without resorting to invasive or poorly-understood modifications.

On the kernel side, real-world kernel initialization time for the Headless Server profile fell from approximately 2.0s (stock) to approximately 0.81s — a reduction of roughly 59% — achieved through three staged removals (tracing subsystem, VideoCore/VCHIQ display stack, USB/PCIe) common in spirit to the “systematic removal of unused kernel subsystems” objective stated in Section 1. Beyond the raw numbers, the investigation surfaced a genuine, previously undocumented-to-the-author upstream kernel lock-contention issue (Section 5.3) affecting tracing-related initcalls.

While, on the other hand, the Interactive Unit profile saw improvements from base 2.65s to 1.07s, which is again roughly 59% reduction in the time spent in the kernel space during the boot process.

On the userspace side, disabling the `cloud-init` provisioning service group produced this project’s first major userspace-track improvement for both profiles, by removing an entire three-step chain (and its upstream filesystem-check dependents) from the critical path entirely. For the Headless profile, investigation of the remaining `NetworkManager-wait-online.service` cost revealed it was dominated by an unmanaged, permanently-disconnected `eth0` interface unconditionally tracked by NetworkManager’s startup-completion logic — not by WPA-Enterprise authentication, which was independently measured at under 10ms. Migrating to `systemd-networkd` with `wpa_supplicant`, with `eth0` explicitly excluded from device management by construction, produced this study’s largest single userspace-track improvement, followed by a further, smaller gain from disabling three confirmed off-critical-chain optional services (Bluetooth, avahi, keyboard-setup).

For the Interactive Unit profile, NetworkManager was retained for its GUI configuration convenience, with the same `eth0` exclusion applied directly rather than discovered through the same investigative process.

Throughout, the project's core methodological commitment — one measurable change per stage, ten repeated boot iterations per measurement, and explicit before/after comparison — proved essential not only for producing defensible numbers, but for correctly diagnosing several unexpected findings (the tracing lock-contention chain, an early DNS resolution failure during network-stack experimentation, and the `eth0` carrier-wait artifact behind the apparent network-readiness cost) that a less disciplined approach would likely have misattributed or left uncorrected.

10.1 Future Work

Several extensions to this work remain open:

- **iwd as a further network-stack data point.** The corrected network-stack investigation (Section 8) migrated from NetworkManager to `systemd-networkd` with `wpa_supplicant`; Intel's `iwd` was identified early in this project as a compatible, potentially lighter-weight alternative to `wpa_supplicant` under `systemd-networkd`, but was not tested. Given how strong and stable the `wpa_supplicant`-based result already is (Section 8), this is a lower-priority, completeness-oriented follow-up rather than an expected further improvement.
- **Serial console instrumentation.** A serial console adapter was not used in this project's final methodology; early boot debugging relied on `dmesg/journalctl` captured post-boot rather than a live serial connection (Section 3.2). A serial console would allow capture of kernel messages from the earliest boot stages, before the point at which `dmesg` is currently able to retrieve them, and is a candidate addition for a future iteration of this methodology.
- **BeagleBone validation.** Another similar embedded platform, Beaglebone would be an interesting addition to this project.
- **Root cause of the .34/.39 `gpio_led_driver_init` version dependence.** As documented in Section 5.3, `gpio_led_driver_init`'s appearance in the initcall log was found to differ between kernel point-versions `6.18.34+rpt-rpi-v8` and `6.18.39+rpt-rpi-v8` independent of tracing configuration, despite a full configuration diff between the two showing no functional difference. This was not investigated further within this study — version was instead fixed as a controlled variable (both profiles pinned to `.34`) — but the underlying cause (a kernel source or firmware change between point-versions not captured by `.config` alone) remains open and could be resolved by diffing the kernel source trees directly, or bisecting across intermediate point-releases if available.
- **Upstream tracing fix verification.** A kernel patch introducing a `trace_async_init` boot parameter (Section 5.3) — which moves the contended `setup_boot_kprobe_events()` call onto an asynchronous workqueue — was identified in upstream mailing list discussion dated January 2026, after this project's kernel version (`rpi-6.18.y`) was branched. Whether this patch has since landed in a later Raspberry Pi fork branch (e.g. `rpi-6.19.y`) is a natural, low-cost follow-up: a single `grep -r "trace_async_init" kernel/trace/` against that branch's source would confirm its presence without requiring a full rebuild or re-test cycle.
- **Optional-service footprint quantification.** Section 8's off-critical-chain removals (Bluetooth, avahi, keyboard-setup) were confirmed to have no boot-time effect, as expected; a follow-up memory/CPU footprint measurement (rather than boot-time measurement) would complete the resource-constrained-embedded-board case for their removal.

- **Home/consumer Wi-Fi as a third authentication-type data point.** Section 8 only consist of enterprise (WPA-Enterprise/RADIUS) network; a fixed consumer-router WPA2/WPA3-PSK network — arguably the most common real-world deployment target — was identified during this project as a useful third comparison point but not yet captured.
- **BSSID pinning trade-off.** Pinning wlan0 to a known BSSID was found to reduce Wi-Fi scan/association time further (Section 8), at the cost of disabling automatic roaming/failover to another access point broadcasting the same SSID. This is noted as a viable further optimization for a stationary deployment, but was not adopted as the default configuration given the reliability trade-off on campus-managed infrastructure.
- **Further headless kernel subsystem removal (audio, Bluetooth) — deferred, not ruled out.** Beyond the subsystems removed in Section 7.1, audio (CONFIG_SND_*) and Bluetooth (CONFIG_BT_*) were both identified as eligible removal candidates for the Headless Server profile under the eligibility criteria established in Section 6 — neither is required by a display-less, audio-less deployment. However, neither appeared as a measurable entry in the baseline or Stage 3 instrumented initcall data (Section 5, Section 7.1): their cost, if any, falls below the threshold of what this study’s methodology can reliably distinguish from run-to-run noise. Consistent with this project’s evidence-driven approach — subsystems are prioritized for removal by measured cost, not simply by eligibility (Section 6) — these were not pursued further, since the effort of an additional build/test stage was judged not to be justified by an as-yet-unmeasured, likely small return. Explicitly re-profiling with a lower detection threshold, or measuring memory/footprint impact rather than boot-time impact (as with the off-critical-chain userspace services in Section 8), is left as a possible future stage should a stricter footprint requirement ever apply.

11 References

1. Raspberry Pi Ltd. *raspberrypi/linux* — Raspberry Pi Foundation’s fork of the Linux kernel. <https://github.com/raspberrypi/linux>
2. Yaxiong Tian, “[PATCH v4 0/5] Tracing: Accelerate Kernel Boot by Asynchronizing,” Linux kernel mailing list, January 2026. Permalink: <https://lore.kernel.org/all/20260128125117.1704853-1-tianyaxiong@xxxxxxxxxxx/>
3. *systemd-analyze(1)* — man page and documentation for boot-time analysis, kernel/userspace timing, blame, and critical-chain subcommands. <https://www.freedesktop.org/software/systemd/man/systemd-analyze.html>
4. *systemd-networkd(8)*, *systemd-networkd-wait-online.service(8)* — network configuration manager and online-readiness unit documentation. <https://www.freedesktop.org/software/systemd/man/systemd-networkd.html>
5. *NetworkManager(8)*, *nm-online(1)* — NetworkManager daemon and startup-online-wait utility documentation.
6. ConnMan Project. *Connection Manager* documentation and source. <https://01.org/connman>
7. Tailscale Inc. *Tailscale documentation* — *tailscaled*, MagicDNS, Tailscale SSH, and Linux network-manager compatibility notes. <https://tailscale.com/kb/>
8. Broadcom Inc. / Cypress Semiconductor. *brcmfmac* driver and *brcmfmac43455-sdio* firmware documentation, as used on the Raspberry Pi 4 Model B wireless interface.

9. Raspberry Pi Ltd. *Raspberry Pi Documentation* — Raspberry Pi OS, `config.txt`, `cmdline.txt`, bootloader/EEPROM recovery, and board-specific reference. <https://www.raspberrypi.com/documentation/>
10. Debian Project. *Debian NTP Pool* — `*.debian.pool.ntp.org` time synchronization servers, referenced during NTP/clock-synchronization troubleshooting (Section 8).
11. Linux kernel documentation. `Documentation/admin-guide/kernel-parameters.txt` — reference for `initcall_debug`, `printk.time`, `loglevel`, and related boot parameters used throughout Section 3 and Section 5.